



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY

实验报告

开课学期: 2025 春季
课程名称: 计算机网络
实验名称: 协议栈设计与实现
学生班级: 5
学生学号: 220110515
学生姓名: 金正达
评阅教师:
报告成绩:

实验与创新实践教育中心制

2025 年 3 月

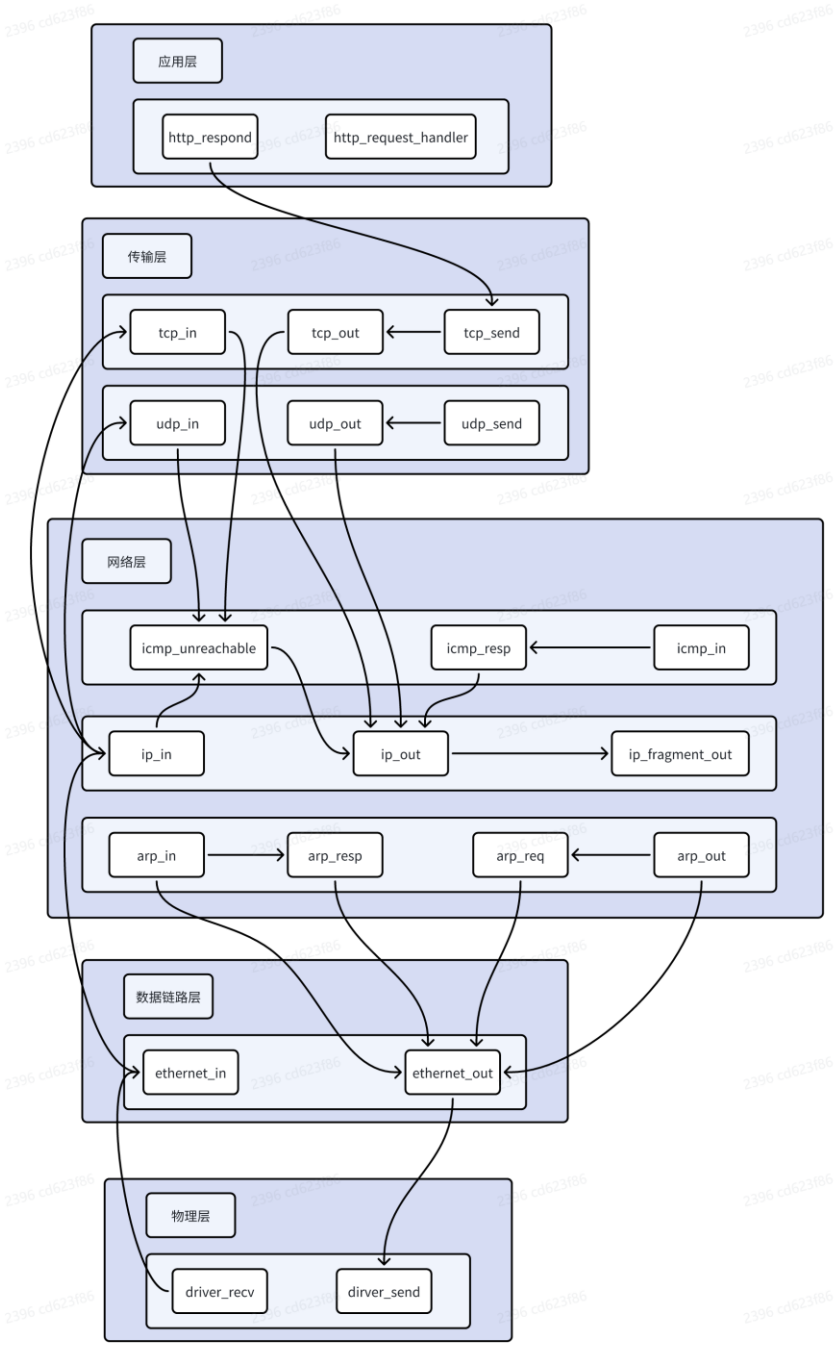
一、 协议实现详述

(注意不要完全照搬实验指导书上的内容，请根据你自己的设计方案来填写
图文并茂地描述实验实现的所有功能和详细的设计方案及实验过程中的特色部分。)

1. 请给出协议栈实验的整体流程图

(绘制协议栈实验的整体流程图，涵盖协议栈接收和发送主要步骤，包括 Eth 接收 / 发送、
ARP 处理、IP 接收 / 发送、ICMP 处理、UDP 接收 / 发送、TCP 接收 / 发送以及 web 服
务器请求处理等步骤，并标注主要函数调用关系。)

协议栈整体流程图：



2. Eth 协议详细设计

(描述以太网 (Eth) 协议的数据封装与解封装过程等。)

➤ ethernet_in 函数用于处理一个收到的数据包:

对于收到的数据包, 先检查其数据长度, 如果数据长度小于以太网包头长度, 则放弃处理:

```
if (buf->len < sizeof(ether_hdr_t)) {
    return;
}
```

如果满足长度要求, 则进行下一步处理, 移除该以太网数据包的包头:

```
ether_hdr_t *hdr = (ether_hdr_t *)buf->data;
buf_remove_header(buf, sizeof(ether_hdr_t));
```

最后, 向上层传递数据包, 按照对应协议和物理地址传递数据包:

```
net_in(buf, swap16(hdr->protocol16), hdr->src);
```

➤ ethernet_out 函数用于处理一个要发送的数据包:

对于要发送的数据包, 首先检查其长度, 如果长度小于以太网最小传输单元长度 46, 就对数据进行填充:

```
if (buf->len < ETHERNET_MIN_TRANSPORT_UNIT) {
    buf_add_padding(buf, ETHERNET_MIN_TRANSPORT_UNIT - buf->len);
}
```

接着, 为数据包添加以太网包头:

```
buf_add_header(buf, sizeof(ether_hdr_t));
```

然后, 将发送目标的 mac 地址、发送源的 mac 地址、发送的协议写入发送数据的包头中:

```
ether_hdr_t *hdr = (ether_hdr_t *)buf->data;
memcpy(hdr->dst, mac, NET_MAC_LEN);
memcpy(hdr->src, net_if_mac, NET_MAC_LEN);
hdr->protocol16 = swap16(protocol);
```

最后, 发送该数据包:

```
driver_send(buf);
```

3. ARP 协议详细设计

(描述 ARP 请求/响应处理逻辑、ARP 表项的更新机制等。)

➤ arp_req 函数用于发送一个 ARP 请求:

初始化缓冲区:

```
buf_init(&txbuf, sizeof(arp_pkt_t));
```

根据协议规范, 填写 ARP 报头, 使用初始 arp 包初始化, 再将目标 ip 地址填入:

```
arp_pkt_t *pkt = (arp_pkt_t *)txbuf.data;
memcpy(pkt, &arp_init_pkt, sizeof(arp_pkt_t));
memcpy(pkt->target_ip, target_ip, NET_IP_LEN);
```

随后，填写 ARP 操作类型为 ARP 请求：

```
pkt->opcode16 = swap16(ARP_REQUEST);
```

设置完后，将数据包使用 ARP 协议广播发送：

```
ethernet_out(&txbuf, ether_broadcast_mac, NET_PROTOCOL_ARP);
```

➤ arp_out 函数用于处理一个需要发送的数据包：

根据 IP 地址查找 ARP 表：

```
uint8_t *mac = (uint8_t *)map_get(&arp_table, ip);
```

如果在表中找到了对应的表项，就将该数据包以检索到的 mac 地址为目标进行发送：

```
if (mac != NULL) {
    ethernet_out(buf, mac, NET_PROTOCOL_ARP);
    return;
}
```

如果表中没有找到对应的表项，就进一步查找 ARP 缓存，如果没有在缓存中找到对应项，就将当前 ip 和数据包存入 ARP 缓存，再发送一个该 ip 的 ARP 请求来获取该 ip 的 mac 地址：

```
if (map_get(&arp_buf, ip) == NULL) {
    map_set(&arp_buf, ip, buf);
    arp_req(ip);
}
```

➤ arp_in 函数用于处理一个收到的数据包：

首先判断收到的数据包的长度，如果长度小于 ARP 头的长度，就将该数据包丢弃，不进行处理：

```
if (buf->len < sizeof(arp_pkt_t)) {
    return;
}
```

接着判断 ARP 头是否符合协议规范，即判断硬件类型是不是以太网类型，协议类型是不是 IP 协议，硬件地址长度和协议地址长度是不是符合要求：

```
arp_pkt_t *pkt = (arp_pkt_t *)buf->data;
if (pkt->hw_type16 != swap16(ARP_HW_ETHER) || pkt->pro_type16 != swap16(NET_PROTOCOL_IP) ||
    pkt->hw_len != NET_MAC_LEN || pkt->pro_len != NET_IP_LEN) {
    return;
}
```

随后更新 ARP 表项，在发送方的 ip 和 mac 之间建立映射：

```
map_set(&arp_table, pkt->sender_ip, pkt->sender_mac);
```

最后，查看 ARP 缓存中是否存在发送方 ip 对应的缓存，如果存在，就向该 ip 对应的 mac 地址发送检索到的缓存数据包，然后在 ARP 缓存中删除该 ip 对应的缓存；如果不存在，就判断接收到的数据的 ARP 类型是不是 ARP 请求，并且目标 ip 是不是本机 ip，若是，就使用发送方的 ip 和 mac 地址进行一个 ARP 响应：

```
buf_t *buf_cache = (buf_t *)map_get(&arp_buf, pkt->sender_ip);
if (buf_cache != NULL) {
    ethernet_out(buf_cache, pkt->sender_mac, NET_PROTOCOL_IP);
    map_delete(&arp_buf, pkt->sender_ip);
} else if (pkt->opcode16 == swap16(ARP_REQUEST) && memcmp(pkt->target_ip, net_if_ip, NET_IP_LEN) == 0) {
    arp_resp(pkt->sender_ip, pkt->sender_mac);
}
```

➤ arp_resp 函数用于发送一个 ARP 响应；

首先，初始化缓冲区：

```
buf_init(&txbuf, sizeof(arp_pkt_t));
```

接着，根据协议规范，填写 ARP 报头，使用初始 arp 包初始化，填入发送的目标 ip 和 mac 地址：

```
arp_pkt_t *pkt = (arp_pkt_t *)txbuf.data;
memcpy(pkt, &arp_init_pkt, sizeof(arp_pkt_t));
memcpy(pkt->target_ip, target_ip, NET_IP_LEN);
memcpy(pkt->target_mac, target_mac, NET_MAC_LEN);
```

设置 ARP 操作类型为 ARP 回复：

```
pkt->opcode16 = swap16(ARP_REPLY);
```

最后，将该数据发送至对应 mac 地址：

```
ethernet_out(&txbuf, target_mac, NET_PROTOCOL_ARP);
```

4. IP 协议详细设计

(描述 IP 数据包的封装与解封装、IP 数据包的分片、校验和计算等。)

➤ ip_init 函数用于处理一个收到的数据包：

首先，判断数据包的长度，如果小于 IP 头部的长度就舍弃，不进行处理：

```
if (buf->len < sizeof(ip_hdr_t)) {
    return;
}
```

接着，检测 IP 报头的字段，检查协议版本是否为 IPv4、总长度字段是否小于或等于收到的数据包长度、目标 IP 地址是否为本机 IP 地址：

```
ip_hdr_t *ip_hdr = (ip_hdr_t *)buf->data;
if (ip_hdr->version != IP_VERSION_4 || swap16(ip_hdr->total_len16) > buf->len || memcmp(ip_hdr->dst_ip, net_if_ip, NET_IP_LEN) != 0) {
    return;
}
```

然后，计算 IP 头部校验和，先保存原来的校验和字段，将其置为 0 后计算校验和，如果计算所得校验和与原始校验和不同，说明传输发生错误，舍弃报文不进行处理，否则，恢复原来的校验和字段：

```
uint16_t hdr_checksum = ip_hdr->hdr_checksum16;
ip_hdr->hdr_checksum16 = 0;
uint16_t checksum = checksum16((uint16_t *)ip_hdr, sizeof(ip_hdr_t));
if (hdr_checksum != checksum) {
    return;
}
ip_hdr->hdr_checksum16 = hdr_checksum;
```

然后，检查接收到的数据包的长度大于 IP 头部的总长度字段，如果大于，说明头部进行过填充，需要去除填充字段：

```
if (buf->len > swap16(ip_hdr->total_len16)) {
    buf_remove_padding(buf, buf->len - swap16(ip_hdr->total_len16));
}
```

最后，去除 IP 头部，使用 net_in 将数据传递给上层，判断是否发送成功，如果未成功，表明遇到不能识别的协议类型，此时为数据重新加上 IP 头部，恢复 IP 头部的数据，发送发出 ICMP 协议报文表明 ICMP 协议不可达：

```
buf_remove_header(buf, sizeof(ip_hdr_t));

if (net_in(buf, ip_hdr->protocol, ip_hdr->src_ip) < 0) {
    buf_add_header(buf, sizeof(ip_hdr_t));
    memcpy(buf->data, ip_hdr, sizeof(ip_hdr_t));
    icmp_unreachable(buf, ip_hdr->src_ip, ICMP_CODE_PROTOCOL_UNREACH);
}
```

➤ checksum16 函数用于计算校验和：

首先，初始化累加和，按 16 位字节序列累加数据，每次处理 2 个字节，如果剩余长度为奇数，将最后 1 个字节左移 8 位作为高字节处理：

```
uint32_t sum = 0;

while(len > 0) {
    if (len == 1) {
        sum += *(uint8_t *)data << 8;
        break;
    } else {
        sum += swap16(*data);
        data++;
        len -= 2;
    }
}
```

然后，处理进位，将 32 位累加结果的高 16 位与低 16 位相加，直到高 16 位为 0：

```
while((sum >> 16) != 0) {
    sum = (sum >> 16) + (sum & 0xFFFF);
}
```

最后，取反并转换字节序返回：

```
return swap16(~(sum & 0xFFFF));
```

➤ ip_out 函数用于处理一个要发送的 IP 数据包：

首先，定义 IP 协议最大负载报长和初始偏移量：

```
int max_payload_len = 1480;
int offset = 0;
```

接着，循环判断当前数据包长度是否大于 IP 协议最大负载报长，如果大于，说明需要进行分片，分片过程为：先初始化一个缓存，大小为最大负载报长，然后将该大小的数据包复制进入缓存，然后移除缓存的 IP 头部，调用 `ip_fragment_out` 将其发出，再重新计算偏移值：

```
while(buf->len > max_payload_len) {
    buf_t ip_buf;
    buf_init(&ip_buf, max_payload_len);
    memcpy(ip_buf.data, buf->data, max_payload_len);
    buf_remove_header(buf, max_payload_len);

    ip_fragment_out(&ip_buf, ip, protocol, id, offset, IP_MORE_FRAGMENT);

    offset += max_payload_len / 8;
}
```

最后，如果还有数据包未超过最大负载报长或者一开始就没超过，直接初始化一个该报文长度的缓存，将其使用 `ip_fragment_out` 发出，最后将全局变量 `id` 的值自增一：

```
buf_t ip_buf;
buf_init(&ip_buf, buf->len);
memcpy(ip_buf.data, buf->data, buf->len);
ip_fragment_out(&ip_buf, ip, protocol, id, offset, 0);
id++;
```

➤ `ip_fragment_out` 函数用于处理一个要发送的 ip 分片：
首先，增加数据包的 IP 头部空间：

```
buf_add_header(buf, sizeof(ip_hdr_t));
ip_hdr_t *ip_hdr = (ip_hdr_t *)buf->data;
```

接着，填写 IP 头部相关信息，包括协议版本、首部长度、服务类型、存活时间等字段：

```
ip_hdr->version = IP_VERSION_4;
ip_hdr->hdr_len = 5;
ip_hdr->tos = 0;
ip_hdr->total_len16 = swap16(buf->len);
ip_hdr->ttl = 64;
ip_hdr->id16 = swap16(id);
ip_hdr->flags_fragment16 = swap16(offset | mf);
ip_hdr->protocol = protocol;
memcpy(ip_hdr->dst_ip, ip, NET_IP_LEN);
memcpy(ip_hdr->src_ip, net_if_ip, NET_IP_LEN);
```

然后，计算校验和，填写校验和字段，再将报文调用 `arp_out` 发出：

```
ip_hdr->hdr_checksum16 = 0;
uint16_t checksum = checksum16((uint16_t *)ip_hdr, sizeof(ip_hdr_t));
ip_hdr->hdr_checksum16 = checksum;

arp_out(buf, ip);
```

5. ICMP 协议详细设计

(解释如何处理 ICMP 请求和响应, 以及如何利用 ICMP 报文进行网络故障诊断等。)

➤ icmp_in 函数用于处理一个收到的数据包:

首先, 判断数据包的长度, 如果小于 ICMP 头部的长度就舍弃, 不进行处理:

```
if (buf->len < sizeof(icmp_hdr_t)) {
    return;
}
```

然后, 查看该 ICMP 报文的类型, 如果是回显请求类型, 直接调用 icmp_resp 将其发送到目标 IP 地址:

```
icmp_hdr_t *icmp_hdr = (icmp_hdr_t *)buf->data;
if (icmp_hdr->type == ICMP_TYPE_ECHO_REQUEST) {
    icmp_resp(buf, src_ip);
}
```

➤ icmp_resp 函数用于发送 ICMP 响应报文:

首先, 初始化缓存, 拷贝报文数据:

```
buf_init(&txbuf, req_buf->len);
memcpy(txbuf.data, req_buf->data, req_buf->len);
```

接着, 填写报头数据, 主要是填写 ICMP 访问类型为回显请求类型, 并将 code 字段填写为 0, 表示查询报文:

```
icmp_hdr_t *icmp_hdr = (icmp_hdr_t *)txbuf.data;
icmp_hdr->type = ICMP_TYPE_ECHO_REPLY;
icmp_hdr->code = 0;
```

最后, 计算并填写校验和, 然后调用 ip_out 将数据以 ICMP 协议发出:

```
icmp_hdr->checksum16 = 0;
uint16_t checksum = checksum16((uint16_t *)txbuf.data, txbuf.len);
icmp_hdr->checksum16 = checksum;

ip_out(&txbuf, src_ip, NET_PROTOCOL_ICMP);
```

➤ icmp_unreachable 函数用于发送 ICMP 不可达信息:

首先, 初始化 ICMP 缓存, 并拷贝收到报文的数据:

```
buf_init(&txbuf, sizeof(ip_hdr_t) + sizeof(icmp_hdr_t));
memcpy(txbuf.data, recv_buf->data, txbuf.len);
```

接着, 添加 ICMP 头部, 填写 ICMP 头部的字段, 填写 ICMP 类型为不可达, code 字段为输入的 code, 标识符和序号均填写为 0:


```
buf_add_header(&txbuf, sizeof(icmp_hdr_t));
icmp_hdr_t *icmp_hdr = (icmp_hdr_t *)txbuf.data;

icmp_hdr->type = ICMP_TYPE_UNREACH;
icmp_hdr->code = code;
icmp_hdr->id16 = 0;
icmp_hdr->seq16 = 0;
```

最后，计算并填写校验和，调用 ip_out 将其以 ICMP 协议发送：

```
icmp_hdr->checksum16 = 0;
uint16_t checksum = checksum16((uint16_t *)txbuf.data, txbuf.len);
icmp_hdr->checksum16 = checksum;

ip_out(&txbuf, src_ip, NET_PROTOCOL_ICMP);
```

6. UDP 协议详细设计

(描述 UDP 数据包的封装与解封装、UDP 校验和计算等。)

➤ udp_out 函数处理一个要发送的数据包：

首先，为数据包添加 UDP 头部：

```
buf_add_header(buf, sizeof(udp_hdr_t));
udp_hdr_t *udp_hdr = (udp_hdr_t *)buf->data;
```

接着，填写 UDP 头部相关字段，包括源端口地址、目标端口地址、整个数据包长度，并计算填写校验和：

```
udp_hdr->src_port16 = swap16(src_port);
udp_hdr->dst_port16 = swap16(dst_port);
udp_hdr->total_len16 = swap16(buf->len);
udp_hdr->checksum16 = 0;

uint16_t checksum = transport_checksum(NET_PROTOCOL_UDP, buf, net_if_ip, dst_ip);
udp_hdr->checksum16 = checksum;
```

最后，调用 ip_out 将数据包以 IP 协议发出：

```
ip_out(buf, dst_ip, NET_PROTOCOL_UDP);
```

➤ udp_in 函数用于处理一个收到的 udp 数据包：

首先，判断数据包的长度，如果小于 UDP 头部的长度就舍弃，不进行处理；再判断接收到的包长度，如果小于 UDP 首部长度字段给出的长度，也舍弃不进行处理：

```
if (buf->len < sizeof(udp_hdr_t)) {
    return;
}
udp_hdr_t *udp_hdr = (udp_hdr_t *)buf->data;
if (swap16(udp_hdr->total_len16) > buf->len) {
    return;
}
```

接着，重新计算校验和，与原有校验和进行对比，如果不一致，说明传输过程发生错误，需要舍弃：

```
uint16_t udp_checksum = udp_hdr->checksum16;
udp_hdr->checksum16 = 0;
uint16_t checksum = transport_checksum(NET_PROTOCOL_UDP, buf, src_ip, net_if_ip);
if (udp_checksum != checksum) {
    return;
}
udp_hdr->checksum16 = udp_checksum;
```

然后，根据对应端口寻找回调函数：

```
uint16_t dst_port = swap16(udp_hdr->dst_port16);
udp_handler_t *handler = (udp_handler_t *)map_get(&udp_table, &dst_port);
```

最后，如果未找到回调函数，就为数据包添加 IP 报头，再调用 icmp_unreachable 发送 ICMP 不可达请求；如果找到了回调函数，就移去数据包的 UDP 报头，再调用回调函数对数据包进行处理：

```
if (handler == NULL) {
    buf_add_header(buf, sizeof(ip_hdr_t));
    icmp_unreachable(buf, src_ip, ICMP_CODE_PORT_UNREACH);
    return;
} else {
    buf_remove_header(buf, sizeof(udp_hdr_t));
    (*handler)(buf->data, buf->len, src_ip, swap16(udp_hdr->src_port16));
}
```

➤ transport_checksum 函数用于计算 UDP 校验和：

首先，增加 UDP 的伪头部，并将伪头部覆盖的 IP 头部拷贝存储起来，防止被覆盖：

```
buf_add_header(buf, sizeof(peso_hdr_t));

uint8_t backup[sizeof(peso_hdr_t)];
memcpy(backup, buf->data, sizeof(peso_hdr_t));
```

接着，填写 UDP 伪头部相关字段，主要包括源 IP 地址、目标 IP 地址、协议号、UDP 长度等：

```
peso_hdr_t *peso_hdr = (peso_hdr_t *)buf->data;
memcpy(peso_hdr->src_ip, src_ip, NET_IP_LEN);
memcpy(peso_hdr->dst_ip, dst_ip, NET_IP_LEN);
peso_hdr->placeholder = 0;
peso_hdr->protocol = protocol;
peso_hdr->total_len16 = swap16(buf->len - sizeof(peso_hdr_t));
```

然后，调用 checksum16 计算校验和：

```
uint16_t checksum = checksum16((uint16_t *)buf->data, buf->len);
```

最后，恢复暂存的 IP 头部，去除添加的 UDP 伪头部，返回校验和：

```
memcpy(buf->data, backup, sizeof(peso_hdr_t));
buf_remove_header(buf, sizeof(peso_hdr_t));

return checksum;
```

7. TCP 协议详细设计

(描述 TCP 连接的建立与关闭过程(三次握手、四次挥手)等。解释如何处理 TCP 数据包的确认、连接状态等问题。)

➤ tcp_out 函数用于填写 TCP 报文头并发送:

首先, 为报文添加 TCP 报头:

```
buf_add_header(buf, sizeof(tcp_hdr_t));
tcp_hdr_t *hdr = (tcp_hdr_t *)buf->data;
```

接着, 填写 TCP 头部相关字段, 设置窗口大小、填写源端口和目标端口, 设置 sequence number 和 ack number 等:

```
hdr->win = swap16(TCP_MAX_WINDOW_SIZE);
hdr->uptr = 0;
hdr->src_port16 = swap16(src_port);
hdr->dst_port16 = swap16(dst_port);
hdr->seq = swap32(tcp_conn->seq);
hdr->ack = swap32(tcp_conn->ack);
hdr->doff = (sizeof(tcp_hdr_t) / 4) << 4;
hdr->flags = flags;
```

然后, 计算并填写校验和:

```
hdr->checksum16 = 0;
hdr->checksum16 = transport_checksum(NET_PROTOCOL_TCP, buf, net_if_ip, dst_ip);
```

最后, 调用 ip_out 将报文以 TCP 协议发出:

```
ip_out(buf, dst_ip, NET_PROTOCOL_TCP);
```

➤ tcp_in 函数用于处理一个收到的 TCP 数据包:

首先, 检查收到数据包的长度, 如果小于 TCP 头部长度就丢弃不处理:

```
if (buf->len < sizeof(tcp_hdr_t))
    return;

tcp_hdr_t *hdr = (tcp_hdr_t *)buf->data;
```

接着, 计算校验和, 如果和头部携带的校验和不同则传输过程发生错误, 丢弃不处理:

```
uint16_t checksum = hdr->checksum16;
hdr->checksum16 = 0;
if (transport_checksum(NET_PROTOCOL_TCP, buf, src_ip, net_if_ip) != checksum)
    return;
```

随后, 获取连接相关信息, 查找并创建 TCP 连接:

```
uint8_t *remote_ip = src_ip;
uint16_t remote_port = swap16(hdr->src_port16);
uint16_t host_port = swap16(hdr->dst_port16);
tcp_conn_t *tcp_conn = tcp_get_connection(remote_ip, remote_port, host_port, true);
```

随后，获取接受的数据包的标志位，如果是重置标志，就关闭 TCP 连接，否则继续计算收到报文的序列号和 TCP 首部长度供下面使用：

```
uint8_t recv_flags = hdr->flags;
// 收到RST, 关闭 TCP 连接
if (TCP_FLG_ISSET(recv_flags, TCP_FLG_RST)) {
    tcp_close_connection(remote_ip, remote_port, host_port);
    return;
}

uint32_t remote_seq = swap32(hdr->seq);
uint32_t tcp_hdr_sz = (hdr->doff >> 4) * 4;
```

然后，根据当前 TCP 连接状态进行不同的处理，并填写回复报文的标志部分：

LISTEN 状态，先检查是否为 SYN 报文，如果不是则丢弃不处理。接着初始化 TCP 连接上下文的序列号，填写确认号、回复标志，最后转移状态为 SYN_RECEIVED 状态：

```
case TCP_STATE_LISTEN:
    // TODO: 仅在收到连接报文时 (SYN报文) 才做出处理, 否则直接返回
    if (!TCP_FLG_ISSET(recv_flags, TCP_FLG_SYN)) {
        return;
    }
    // TODO: 初始化 TCP 连接上下文 (tcp_conn结构体) 的seq字段
    tcp_conn->seq = tcp_generate_initial_seq();
    // TODO: 填写 TCP 连接上下文 (tcp_conn结构体) 的ack字段
    tcp_conn->ack = remote_seq + 1;
    // TODO: 填写回复标志 send_flags
    send_flags = TCP_FLG_ACK | TCP_FLG_SYN;
    // TODO: 进行状态转移
    tcp_conn->state = TCP_STATE_SYN_RECEIVED;
    break;
```

SYN_RECEIVED 状态，先判断是否为 ACK 报文，如果不是则丢弃不进行处理。然后直接进行状态转移至 ESTABLISHED 状态：

```
case TCP_STATE_SYN_RECEIVED:
    // TODO: 仅在收到确认报文时 (ACK报文) 才做出处理, 否则直接返回
    if (!TCP_FLG_ISSET(recv_flags, TCP_FLG_ACK)) {
        return;
    }
    // TODO: 进行状态转移
    tcp_conn->state = TCP_STATE_ESTABLISHED;
    break;
```

ESTABLISHED 状态，首先检查序列号是否和当前 TCP 连接确认号一致，如果不一致，就丢弃，初始化缓存并发送重复的 ACK 报文。下面计算收到的数据长度来更新当前 TCP 连接的 ACK 字段，再判断接受的报文是否携带数据，如果携带就为发送标志增加 ACK 标志，最后判断接受到的是否为 FIN 报文，如果是，就为发送标志添加 ACK 和 FIN 标志，再进行状态转移至 LAST_ACK 状态：

```

case TCP_STATE_ESTABLISHED:
    // 未收到顺序包, 丢弃并发送重复 ACK
    if (remote_seq != tcp_conn->ack) {
        buf_init(&txbuf, 0);
        tcp_out(tcp_conn, &txbuf, host_port, remote_ip, remote_port, TCP_FLG_ACK);
        return;
    }
    // TODO: 计算接收到的数据长度, 更新 ACK
    tcp_conn->ack = remote_seq + bytes_in_flight(buf->len - tcp_hdr_sz, recv_flags);
    // TODO: 如果接收报文携带数据, 则填写回复标志 send_flags 发送ACK
    if (buf->len > tcp_hdr_sz) {
        send_flags |= TCP_FLG_ACK;
    }
    // TODO: 如果收到 FIN 报文, 则增加 send_flags 相应标志位, 并且进行状态转移
    if (TCP_FLG_ISSET(recv_flags, TCP_FLG_FIN)) {
        send_flags |= TCP_FLG_ACK | TCP_FLG_FIN;
        tcp_conn->state = TCP_STATE_LAST_ACK;
    }
    break;

```

LAST_ACK 状态, 先判断该报文是否为 ACK 报文, 如果不是则丢弃不处理。然后直接关闭 TCP 连接:

```

case TCP_STATE_LAST_ACK:
    // TODO: 仅在收到确认报文时 (ACK 报文) 才做出处理, 否则直接返回
    if (!TCP_FLG_ISSET(recv_flags, TCP_FLG_ACK)) {
        return;
    }
    // TODO: 关闭 TCP 连接
    tcp_close_connection(remote_ip, remote_port, host_port);
    break;

```

填写完 TCP 的连接状态和回复报文标志后, 判断当前报文是否携带数据, 如果携带, 就调用 map_get 函数查询 tcp_handler_table 是否有该目的端口号对应的回调函数, 如果有, 就移除报文的 TCP 头部, 调用回调函数处理该报文; 如果没有, 就为报文增加 IP 头部, 再调用 icmp_unreachable 发送端口不可达信息:

```

if (buf->len - tcp_hdr_sz > 0) {
    tcp_handler_t *handler = (tcp_handler_t*)map_get(&tcp_handler_table, &host_port);
    if (handler) {
        buf_remove_header(buf, tcp_hdr_sz);
        (*handler)(tcp_conn, buf->data, buf->len, remote_ip, remote_port);
    } else {
        buf_add_header(buf, sizeof(ip_hdr_t));
        icmp_unreachable(buf, remote_ip, ICMP_CODE_PORT_UNREACH);
    }
}

```

然后, 根据上述填写的发送标志, 判断处理方式, 如果标志位为 0, 表明无需回复。如果发送标志位标识了 ACK, 且已经通过 tcp_send 发送 ACK, 则不进行回复:

```
// 如果无需回复, 则接收逻辑结束
if (send_flags == 0)
    return;
// 如果 send_flags 只标识了 ACK 字段, 并且应用程序已通过 tcp_send() 发送顺带 ACK, 则无需再进行回复
if (bytes_in_flight(0, send_flags) == 0 && tcp_conn->not_send_empty_ack) {
    assert(TCP_FLG_ISSET(send_flags, TCP_FLG_ACK));
    tcp_conn->not_send_empty_ack = 0;
    return;
}
```

最后, 初始化一个新的缓冲区, 以填写的回复标志发送回复报文, 再更新当前 TCP 连接的序列号:

```
// TODO: 初始化一个新的缓冲区, 发送回复报文
buf_init(&txbuf, 0);
tcp_out(tcp_conn, &txbuf, host_port, remote_ip, remote_port, send_flags);
// TODO: 更新序列号
tcp_conn->seq += bytes_in_flight(0, send_flags);
```

8. web 服务器详细设计

(描述 web 服务器的请求处理流程和响应等。)

初始化和监听:

1. 首先调用 net_init 函数初始化协议栈
2. 调用 tcp_open 在 80 端口注册 TCP 监听回调
3. 进入无限循环, 不断调用 net_poll 轮询网络事件

```
int main(int argc, char const *argv[]) {
    if (net_init() == -1) { // 初始化协议栈
        printf("net init failed.");
        return -1;
    }

    tcp_open(HTTP_LISTEN_PORT, http_request_handler); // 注册端口的tcp监听回调

    while (1) {
        net_poll(); // 一次主循环
    }

    return 0;
}
```

请求处理流程:

当收到 HTTP 请求时, 会触发回调函数, 处理流程:

1. 解析 HTTP 请求: 提取 HTTP 方法, 仅支持 GET 请求, 再解析请求的 URL 路径
2. 调用响应函数 http_respond 处理请求并生成响应

```

void http_request_handler(tcp_conn_t *tcp_conn, uint8_t *data, size_t len, uint8_t *src_ip, uint16_t src_port) {
    char method[4];
    char url_path[HTTP_MAX_PATH_LENGTH];

    // 提取 HTTP 方法。目前仅支持 "GET" 请求
    if (sscanf((char *)data, "%3s", method) != 1 || strcmp(method, "GET") != 0)
        return;

    // 获取请求 URL
    int idx = 0;
    int j = 0;
    while (data[idx] != ' ')
        ++idx;
    ++idx;
    while (data[idx] != ' ') {
        url_path[j++] = data[idx++];
    }
    url_path[j] = '\0';

    // 发送响应
    http_respond(tcp_conn, url_path, HTTP_LISTEN_PORT, src_ip, src_port);
}

```

响应生成流程:

http_respond 函数负责生成 HTTP 响应:

首先确定文件路径, 如果请求路径是 "/", 则默认访问 "index.html", 其他情况下将请求路径与资源目录拼接, 形成完整文件路径:

```

FILE *file;
char file_path[HTTP_MAX_PATH_LENGTH];
memcpy(file_path, HTTP_RESOURCE_DIR, sizeof(HTTP_RESOURCE_DIR));

// 获取文件路径, 打开文件
if (strcmp(url_path, "/") == 0) { // 如果路径为 "/", 则默认打开 index.html
    strcat(file_path, "/index.html");
} else {
    strcat(file_path, url_path); // 否则, 文件路径为 "${HTTP_RESOURCE_DIR}${url_path}"
}

// 打开文件
file = fopen(file_path, "rb");

```

接着, 处理文件不存在处理 (404 响应), 尝试打开文件, 如果文件不存在, 发送 404 响应:

- 发送 HTTP 状态行: HTTP/1.1 404 Not Found
- 发送连接信息: Connection: Keep-Alive
- 发送内容类型: Content-Type: text/html; charset=utf-8
- 发送内容长度: Content-Length: [长度]
- 发送空行作为响应头与响应体的分隔符
- 发送 404 错误页面的 HTML 内容

```

if (!file) {
    // HTTP 404 响应请求体
    char *not_found_body = "<HTML><TITLE>Not Found</TITLE>\r\n"
                           "The resource specified\r\n"
                           "is unavailable or nonexistent.\r\n"
                           "</BODY></HTML>\r\n";

    /* Step1 : 发送 HTTP 404 请求头 */
    // TODO: 发送 HTTP 状态行
    sprintf(resp_buffer, "HTTP/1.1 404 Not Found\r\n");
    tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);

    // 发送 HTTP 连接信息
    sprintf(resp_buffer, "Connection: Keep-Alive\r\n");
    tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);

    // TODO: 发送 HTTP 内容类型
    sprintf(resp_buffer, "Content-Type: text/html; charset=utf-8\r\n");
    tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);

    // TODO: 发送 HTTP 内容长度
    sprintf(resp_buffer, "Content-Length: %zu\r\n", strlen(not_found_body));
    tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);

    // TODO: 发送 HTTP 响应头与响应体的分隔符
    sprintf(resp_buffer, "\r\n");
    tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);

    // TODO: 发送 HTTP 响应体
    tcp_send(tcp_conn, (uint8_t *)not_found_body, strlen(not_found_body), port, dst_ip, dst_port);
    return;
}

```

否则，处理文件存在处理（200 响应）：

- 发送 HTTP 状态行：HTTP/1.1 200 OK
- 发送连接信息：Connection: Keep-Alive
- 根据文件类型确定并发送 MIME 类型
- 计算并发送文件大小作为内容长度
- 发送空行作为响应头与响应体的分隔符
- 分块读取文件内容并发送

```

// TODO: 发送 HTTP 状态行
sprintf(resp_buffer, "HTTP/1.1 200 OK\r\n");
tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);

// 发送 HTTP 连接信息
sprintf(resp_buffer, "Connection: Keep-Alive\r\n");
tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);

const char *content_type = http_get_mime_type(file_path);
// TODO: 发送 HTTP 内容类型, 根据文件类型设置 MIME 类型
sprintf(resp_buffer, "Content-Type: %s\r\n", content_type);
tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);

fseek(file, 0, SEEK_END);
size_t content_length = ftell(file);
fseek(file, 0, SEEK_SET);
// TODO: 发送 HTTP 内容长度
sprintf(resp_buffer, "Content-Length: %zu\r\n", content_length);
tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);

// TODO: 发送 HTTP 响应头与响应体的分隔符
sprintf(resp_buffer, "\r\n");
tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);

/* Step3 : 发送 HTTP 响应体 */
size_t bytes_read;
while ((bytes_read = fread(resp_buffer, 1, sizeof(resp_buffer), file)) > 0) {
    // TODO: 每次发送读取的文件内容块
    tcp_send(tcp_conn, (uint8_t *)resp_buffer, bytes_read, port, dst_ip, dst_port);
}

```


MIME 类型处理:

服务器通过 `http_get_mime_type` 函数根据文件扩展名返回合适的 MIME 类型:

- `.html/.htm` 返回 `"text/html; charset=utf-8"`
- `.css` 返回 `"text/css"`
- `.jpg/.jpeg` 返回 `"image/jpeg"`
- 其他类型默认返回 `"application/octet-stream"`

```
static inline const char *http_get_mime_type(const char *file_path) {  
    if (strstr(file_path, ".html") || strstr(file_path, ".htm")) {  
        return "text/html; charset=utf-8";  
    } else if (strstr(file_path, ".css")) {  
        return "text/css";  
    } else if (strstr(file_path, ".jpg") || strstr(file_path, ".jpeg")) {  
        return "image/jpeg";  
    }  
    return "application/octet-stream"; // 默认类型  
}
```

二、 实验结果截图及分析

(请展示并详细分析实验结果的截图。可以利用 log 文件、通过 Wireshark 打开的 pcap 文件或 Wireshark 实时捕获的网络报文。)

1. Eth 协议实验结果及分析

(展示以太网帧捕获截图，分析帧的结构和内容是否符合预期。检查目的 MAC 地址、源 MAC 地址、协议类型字段以及数据部分)

能够通过 `eth_in` 测试:

```
• (base) PS C:\Users\Jin_zd\Desktop\Code\Protocol-Stack-Lab\build> ctest -R eth_in  
Test project C:/Users/Jin_zd/Desktop/Code/Protocol-Stack-Lab/build  
Start 1: eth_in  
1/1 Test #1: eth_in ..... Passed    0.16 sec  
  
100% tests passed, 0 tests failed out of 1  
  
Total Test time (real) = 0.17 sec
```

log 文件对比，无不同之处:

能够通过 eth_out 测试:

log 文件对比, 无不同之处:

Wireshark 打开的 in.pcap 文件如下:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.2.15	192.168.163.103	TCP	54	22 → 64289 [ACK] Seq=1 Ack=1 Win=65535 Len=0
2	0.000000	10.0.2.15	192.168.163.103	SSH	106	Server: Encrypted packet (len=52)
3	0.000000	10.0.2.15	192.168.163.103	DNS	84	Standard query 0x96da A www.baidu.com OPT
4	0.000000	10.0.2.15	192.168.163.103	DNS	84	Standard query 0x9759 AAAA www.baidu.com OPT
5	0.000000	10.0.2.15	192.168.163.103	DNS	87	Standard query 0x5a54 AAAA www.a.shifen.com OPT
6	0.000000	10.0.2.15	192.168.163.103	ICMP	98	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (no response found!)
7	0.000000	10.0.2.15	192.168.163.103	TCP	54	[TCP Previous segment not captured] 22 → 64289 [ACK] Seq=261 Ack=261 Win=65535 Len=0
8	0.000000	10.0.2.15	192.168.163.103	SSH	106	Server: Encrypted packet (len=52)
9	0.000000	PCSSystemtec_6c:48:...	11:22:33:44:55:66	ARP	42	Who has 192.168.163.103? Tell 10.0.2.15
10	0.000000	PCSSystemtec_6c:48:...	11:22:33:44:55:66	ARP	42	Who has 192.168.163.103? Tell 10.0.2.15
11	0.000000	10.0.2.15	192.168.163.103	ICMP	98	Echo (ping) request id=0x0003, seq=1/256, ttl=64 (no response found!)
12	0.000000	10.0.2.15	192.168.163.103	DNS	100	Standard query 0x5d43 A connectivity-check.ubuntu.com OPT
13	0.000000	PCSSystemtec_6c:48:...	11:22:33:44:55:66	ARP	42	Who has 192.168.163.103? Tell 10.0.2.15
14	0.000000	10.0.2.15	192.168.163.103	HTTP	141	GET / HTTP/1.1
15	0.000000	66:91:9b:04:ca:64	11:22:33:44:55:66	0x14b0	60	Ethernet II
16	0.000000	f8:85:0f:26:a0:e4	11:22:33:44:55:66	0x1f1f	60	Ethernet II
17	0.000000	1d:6b:a4:32:cb:56	11:22:33:44:55:66	0xe610	60	Ethernet II

选取 Round1，查看具体捕获到的报文：

> Frame 1: 54 bytes on wire (432 bits), 54 bytes captured (432 bits)

▼ Ethernet II, Src: PCSSystemtec_6c:48:4f (08:00:27:6c:48:4f), Dst: 11:22:33:44:55:66 (11:22:33:44:55:66)

▼ Destination: 11:22:33:44:55:66 (11:22:33:44:55:66)
.....0..... = LG bit: Globally unique address (factory default)
.....1..... = IG bit: Group address (multicast/broadcast)
▼ Source: PCSSystemtec_6c:48:4f (08:00:27:6c:48:4f)
.....0..... = LG bit: Globally unique address (factory default)
.....0..... = IG bit: Individual address (unicast)
Type: IPv4 (0x0800)
[Stream index: 0]

▼ Internet Protocol Version 4, Src: 10.0.2.15, Dst: 192.168.163.103
0100 = Version: 4
.... 0101 = Header Length: 20 bytes (5)
> Differentiated Services Field: 0x08 (DSCP: Unknown, ECN: Not-ECT)
Total Length: 40
Identification: 0x06f4 (1780)
> 010. = Flags: 0x2, Don't fragment
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 64
Protocol: TCP (6)
Header Checksum: 0x1bc4 [validation disabled]
[Header checksum status: Unverified]
Source Address: 10.0.2.15
Destination Address: 192.168.163.103
[Stream index: 0]

> Transmission Control Protocol, Src Port: 22, Dst Port: 64289, Seq: 1, Ack: 1, Len: 0

0000 11 22 33 44 55 66 08 00 27 6c 48 4f 08 00 45 08 ~3Duf... 'lHO :E

0010 00 20 06 f4 40 00 40 06 1b c4 0a 00 02 0f c0 a5 -(@ @:

0020 a3 67 00 16 fb 21 4f 43 b0 37 22 ea f8 1f 50 10 -g...lOC 7"...P

0030 ff ff 18 2b 00 00:..

如图所示，
以太网协议层：
源 mac 地址为 08:00:27:6c:48:4f，目的 mac 地址为 11:22:33:44:55:66，
IP 层：
协议类型字段在 IP 层被标识为 Type: IPv4 (0x0800)，
IP 头部加上数据字段与 log 文件中一致，均符合预期。
发送的数据报被正确的去除以太网头部输出在 log 文件中。
其余情况类似，表明 eth_in 实现正确。

Wireshark 打开 out.pcap 文件：

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.163.103	10.0.2.2	TCP	60	22 → 64289 [ACK] Seq=1 Ack=1 Win=65535 Len=0
2	0.000000	192.168.163.103	10.0.2.2	SSH	106	Server: Encrypted packet (len=52)
3	0.000000	192.168.163.103	10.248.98.30	DNS	84	Standard query 0x96da A www.baidu.com OPT
4	0.000000	192.168.163.103	10.248.98.30	DNS	84	Standard query 0x9759 AAAA www.baidu.com OPT
5	0.000000	192.168.163.103	10.248.98.30	DNS	87	Standard query 0x5a54 AAAA www.a.shifen.com OPT
6	0.000000	192.168.163.103	183.232.231.172	ICMP	98	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (no response found!)
7	0.000000	192.168.163.103	10.0.2.2	TCP	60	[TCP Previous segment not captured] 22 → 64289 [ACK] Seq=261 Ack=261 Win=65535 Len=0
8	0.000000	192.168.163.103	10.0.2.2	SSH	106	Server: Encrypted packet (len=52)
9	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	Who has 10.0.2.3? Tell 192.168.163.103
10	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	Who has 10.0.2.4? Tell 192.168.163.103
11	0.000000	192.168.163.103	10.0.2.4	ICMP	98	Echo (ping) request id=0x0003, seq=1/256, ttl=64 (no response found!)
12	0.000000	192.168.163.103	10.248.98.30	DNS	100	Standard query 0x5d43 A connectivity-check.ubuntu.com OPT
13	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	Who has 10.0.2.5? Tell 192.168.163.103
14	0.000000	192.168.163.103	35.224.99.156	HTTP	141	GET / HTTP/1.1
15	0.000000	11:22:33:44:55:66	11:25:45:c2:dc:93	0xb5f8	60	Ethernet II
16	0.000000	11:22:33:44:55:66	a5:8e:1e:83:1f:35	0xb6c9	60	Ethernet II
17	0.000000	11:22:33:44:55:66	ef:8d:35:50:6c:c2	0x1712	60	Ethernet II

Wireshark 打开 demo_out.pcap 文件：

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.163.103	10.0.2.2	TCP	60	22 → 64289 [ACK] Seq=1 Ack=1 Win=65535 Len=0
2	0.000000	192.168.163.103	10.0.2.2	SSH	106	Server: Encrypted packet (len=52)
3	0.000000	192.168.163.103	10.248.98.30	DNS	84	Standard query 0x96da A www.baidu.com OPT
4	0.000000	192.168.163.103	10.248.98.30	DNS	84	Standard query 0x9759 AAAA www.baidu.com OPT
5	0.000000	192.168.163.103	10.248.98.30	DNS	87	Standard query 0x5a54 AAAA www.a.shifen.com OPT
6	0.000000	192.168.163.103	183.232.231.172	ICMP	98	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (no response found!)
7	0.000000	192.168.163.103	10.0.2.2	TCP	60	[TCP Previous segment not captured] 22 → 64289 [ACK] Seq=261 Ack=261 Win=65535 Len=0
8	0.000000	192.168.163.103	10.0.2.2	SSH	106	Server: Encrypted packet (len=52)
9	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	Who has 10.0.2.3? Tell 192.168.163.103
10	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	Who has 10.0.2.4? Tell 192.168.163.103
11	0.000000	192.168.163.103	10.0.2.4	ICMP	98	Echo (ping) request id=0x0003, seq=1/256, ttl=64 (no response found!)
12	0.000000	192.168.163.103	10.248.98.30	DNS	100	Standard query 0x5d43 A connectivity-check.ubuntu.com OPT
13	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	Who has 10.0.2.5? Tell 192.168.163.103
14	0.000000	192.168.163.103	35.224.99.156	HTTP	141	GET / HTTP/1.1
15	0.000000	11:22:33:44:55:66	11:25:45:c2:dc:93	Ethernet II	0xb5f8	60 Ethernet II
16	0.000000	11:22:33:44:55:66	a5:8e:1e:83:1f:35	Ethernet II	0x16c9	60 Ethernet II
17	0.000000	11:22:33:44:55:66	ef:8d:35:50:6c:c2	Ethernet II	0x1712	60 Ethernet II

demo_out 和 out 文件相同。

上述 eth_in 分析中，数据报被去除了以太网头部，在数据链路层被发出时确实加上了头部，表明 eth_out 实现正确。

2. ARP 协议实验结果及分析

(展示 ARP 请求和响应包的捕获截图，分析其请求和响应过程是否正常。检查 ARP 请求中的目标 IP 地址和发送方 MAC 地址，ARP 响应中的目标 MAC 地址。)

能够通过 arp_test 测试：

```

(base) PS C:\Users\Jin_zd\Desktop\Code\Protocol-Stack-Lab\build> ctest -R arp_test
Test project C:/Users/Jin_zd/Desktop/Code/Protocol-Stack-Lab/build
Start 3: arp_test
1/1 Test #3: arp_test ..... Passed    0.20 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) = 0.22 sec

```

demo_log 和 log 的对比没有不同：

```

demo_log > diff -u demo_log log
1 driver opened
2 <===== arp table =====
3 <===== arp buf =====
4
5 Round 01 -----
6 <===== arp table =====
7 <===== arp buf =====
8 192.168.163.10 -> 45 00 00 46 fb 7c 40 00 40 11 77 67 c0 a8 a3 67 c0 a8 a3 0a ae 1b
9
10 Round 02 -----
11 <===== arp table =====
12 192.168.163.10 -> 21:32:43:54:65:06
13 <===== arp buf =====
14
15 Round 03 -----
16 <===== arp table =====
17 192.168.163.10 -> 21:32:43:54:65:06
18 <===== arp buf =====
19
20 Round 04 -----
21 ip_in:
22 mac:21:32:43:54:65:06
23 buf: 45 00 00 9a 88 e4 00 00 40 11 20 ac c0 a8 a3 0a c0 a8 a3 67 00 35 bf 6a 00
24 <===== arp table =====
25 192.168.163.10 -> 21:32:43:54:65:06
26 <===== arp buf =====
27
28 Round 05 -----
29 ip_in:
30 mac:21:32:43:54:65:06
31 buf: 45 00 00 81 88 e5 00 00 40 11 29 c4 c0 a8 a3 0a c0 a8 a3 67 00 35 ae 1b 00
32 <===== arp table =====
33 192.168.163.10 -> 21:32:43:54:65:06
34 <===== arp buf =====
35
36 Round 06 -----
37 <===== arp table =====
38 192.168.163.10 -> 21:32:43:54:65:06
39 <===== arp buf =====
40
41 Round 07 -----
42 ip_in:
43 mac:21:32:43:54:65:06

```

Wireshark 打开 in.pcap:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.163.103	192.168.163.10	DNS	84	Standard query 0x96da A www.baidu.com OPT
2	0.000000	21:32:43:54:65:06	11:22:33:44:55:66	ARP	42	192.168.163.10 is at 21:32:43:54:65:06
3	0.000000	192.168.163.103	192.168.163.10	DNS	84	Standard query 0x9759 AAAA www.baidu.com OPT
4	0.000000	192.168.163.10	192.168.163.103	DNS	168	Standard query response 0x9759 AAAA www.baidu.com CNAME www.a.shifen.com SOA ns1.a.shifen.com OPT
5	0.000000	192.168.163.10	192.168.163.103	DNS	143	Standard query response 0x96da A www.baidu.com CNAME www.a.shifen.com A 183.232.231.174 A 183.232.231.172 OPT
6	0.000000	192.168.163.103	192.168.163.10	DNS	87	Standard query 0x5a54 AAAA www.a.shifen.com OPT
7	0.000000	192.168.163.10	192.168.163.103	DNS	150	Standard query response 0x5a54 AAAA www.a.shifen.com SOA ns1.a.shifen.com OPT
8	0.000000	192.168.163.110	192.168.163.103	ICMP	98	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 11)
9	0.000000	Pixim_34:45:56	11:22:33:44:55:66	ARP	42	192.168.163.110 is at 01:12:23:34:45:56
10	0.000000	1a:94:f0:3c:49:aa	Broadcast	ARP	42	Who has 192.168.163.103? Tell 192.168.163.2
11	0.000000	192.168.163.103	192.168.163.110	ICMP	98	Echo (ping) reply id=0x0001, seq=1/256, ttl=54 (request in 8)
12	0.000000	192.168.163.2	192.168.163.103	SSH	106	Client: Encrypted packet (len=52)
13	0.000000	192.168.163.103	192.168.163.2	TCP	54	22 → 64289 [ACK] Seq=1 Ack=53 Win=65535 Len=0
14	0.000000	192.168.163.103	192.168.163.10	HTTP	141	GET / HTTP/1.1
15	0.000000	192.168.163.10	192.168.163.103	TCP	60	80 → 55428 [ACK] Seq=1 Ack=88 Win=65535 Len=0

Wireshark 打开 out.pcap:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	ARP Announcement for 192.168.163.103
2	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	Who has 192.168.163.10? Tell 192.168.163.103
3	0.000000	192.168.163.103	192.168.163.10	DNS	84	Standard query 0x96da A www.baidu.com OPT
4	0.000000	192.168.163.103	192.168.163.10	DNS	84	Standard query 0x9759 AAAA www.baidu.com OPT
5	0.000000	192.168.163.103	192.168.163.10	DNS	87	Standard query 0x5a54 AAAA www.a.shifen.com OPT
6	0.000000	11:22:33:44:55:66	1a:94:f0:3c:49:aa	ARP	60	192.168.163.103 is at 11:22:33:44:55:66
7	0.000000	192.168.163.103	192.168.163.110	ICMP	98	Echo (ping) reply id=0x0001, seq=1/256, ttl=54
8	0.000000	192.168.163.103	192.168.163.2	TCP	60	22 → 64289 [ACK] Seq=1 Ack=53 Win=65535 Len=0
9	0.000000	192.168.163.103	192.168.163.10	HTTP	141	GET / HTTP/1.1

在 in 中，第一条报文发送了 DNS 请求，但因为没有对应 IP 地址 192.168.163.10 的 MAC 地址，不能发送成功，于是该数据包被缓存起来。

随后发送了 ARP 广播请求获取对应 IP 的 MAC 地址，即 out 中的第 1 条报文发送无回报的 ARP 报文广播自己 IP 地址 192.168.163.103，out 中的第 2 条报文广播请求对应的 MAC 地址，两条 ARP 报文的源 MAC 地址均为 11:22:33:44:55:66，即 192.168.163.10 对应的 MAC 地址。由 in 中的第 2 条报文可知，192.168.163.103 收到了 ARP 响应报文，获得了 192.168.163.10 的 MAC 地址为 21:32:43:54:65:06。

随后，两者之间可以进行正常的通信。

上述分析表明 ARP 相关函数的实现是正确的。

3. IP 协议实验结果及分析

(展示 IP 数据包(包括分片)的捕获截图，分析 IP 头部字段的正确性。检查版本号、首部长度、总长度、标识、标志位、片偏移、TTL、协议类型等字段，同时分析分片机制是否准确。)

能够通过 ip_test 测试:

```
(base) PS C:\Users\Jin_zd\Desktop\Code\Protocol-Stack-Lab\build> ctest -R ip_test
Test project C:/Users/Jin_zd/Desktop/Code/Protocol-Stack-Lab/build
  Start 4: ip_test
1/1 Test #4: ip_test ..... Passed    0.05 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) = 0.06 sec
```

demo_log 与 log 文件对比，没有不同:


```

Testing 1 data > ip_frag_test > log
1 driver opened
2 <===== arp table =====
3 <===== arp buf =====
4
5 Round 01 -----
6 <===== arp table =====
7 <===== arp buf =====
8 192.168.163.10 -> 45 00 00 46 00 00 00 40 11 b2 e4 c0 a8 a3 67 c0 a8 a3 0a ae 1b
9
10 Round 02 -----
11 <===== arp table =====
12 192.168.163.10 -> 21:32:43:54:65:06
13 <===== arp buf =====
14
15 Round 03 -----
16 <===== arp table =====
17 192.168.163.10 -> 21:32:43:54:65:06
18 <===== arp buf =====
19
20 Round 04 -----
21 <===== arp table =====
22 192.168.163.10 -> 21:32:43:54:65:06
23 <===== arp buf =====
24
25 Round 05 -----
26 udp_in:
27   src_ip:192.168.163.10
28   buf: 00 35 ae 1b 00 6d bb f0 96 da 81 80 00 01 00 03 00 00 01 03 77 77 77
29 <===== arp table =====
30 192.168.163.10 -> 21:32:43:54:65:06
31 <===== arp buf =====
32
33 Round 06 -----
34 <===== arp table =====
35 192.168.163.10 -> 21:32:43:54:65:06
36 <===== arp buf =====
37
38 Round 07 -----
39 udp_in:
40   src_ip:192.168.163.10
41   buf: 00 35 84 9f 00 74 72 81 5a 54 81 80 00 01 00 00 01 00 01 03 77 77 77
42 <===== arp table =====
43 <===== arp buf =====

```

能够通过 ip_frag_test 测试:

```

(base) PS C:\Users\Jin_zd\Desktop\Code\Protocol-Stack-Lab\build> ctest -R ip_frag_test
Test project C:/Users/Jin_zd/Desktop/Code/Protocol-Stack-Lab/build
Start 5: ip_frag_test
1/1 Test #5: ip_frag_test ..... Passed    0.03 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) =  0.03 sec

```

Demo_log 与 log 文件对比, 没有不同:

```

Testing 1 data > ip_frag_test > log
1 arp_out:
2 ip:192.168.163.103
3 buf: 45 00 05 dc 00 00 20 00 40 06 8c fc c0 a8 a3 67 c0 a8 a3 67 41 6c 69 63
4 arp_out:
5 ip:192.168.163.103
6 buf: 45 00 05 dc 00 00 20 b9 40 06 8c 43 c0 a8 a3 67 c0 a8 a3 67 74 20 6f 6e
7 arp_out:
8 ip:192.168.163.103
9 buf: 45 00 05 dc 00 00 21 72 40 06 8b 8a c0 a8 a3 67 c0 a8 a3 67 65 72 20 6c
10 arp_out:
11 ip:192.168.163.103
12 buf: 45 00 02 6c 00 00 02 2b 40 06 ae 41 c0 a8 a3 67 c0 a8 a3 67 20 65 61 74
13

```

Wireshark 打开 in.pcap 文件:

1	0.000000	192.168.163.103	192.168.163.10	DNS	84 Standard query 0x96da A www.baidu.com OPT
2	0.000000	21:32:43:54:65:06	11:22:33:44:55:66	ARP	42 192.168.163.10 is at 21:32:43:54:65:06
3	0.000000	192.168.163.103	192.168.163.10	DNS	84 Standard query response 0x9759 AAAA www.baidu.com OPT
4	0.000000	192.168.163.10	192.168.163.103	DNS	168 Standard query response 0x9759 AAAA www.baidu.com CHANE www.a.shifen.com SOA ns1.a.shifen.com OPT
5	0.000000	192.168.163.10	192.168.163.103	DNS	143 Standard query response 0x96da A www.baidu.com CHANE www.a.shifen.com A 183.232.231.174 A 183.232.231.172 OPT
6	0.000000	192.168.163.103	192.168.163.10	DNS	87 Standard query 0x5a54 AAAA www.a.shifen.com OPT
7	0.000000	192.168.163.10	192.168.163.103	DNS	150 Standard query response 0x5a54 AAAA www.a.shifen.com SOA ns1.a.shifen.com OPT
8	0.000000	192.168.163.110	192.168.163.103	ICMP	98 Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 11)
9	0.000000	Pixim_34:45:56	11:22:33:44:55:66	ARP	42 192.168.163.110 is at 01:12:23:34:45:56
10	0.000000	1a:94:f0:3c:49:aa	Broadcast	ARP	42 Who has 192.168.163.103? Tell 192.168.163.2
11	0.000000	192.168.163.103	192.168.163.110	ICMP	98 Echo (ping) reply id=0x0001, seq=1/256, ttl=64 (request in 8)
12	0.000000	192.168.163.2	192.168.163.103	SSH	106 Client: Encrypted packet (len=52)
13	0.000000	192.168.163.103	192.168.163.2	TCP	54 22 → 64289 [ACK] Seq=1 Ack=53 Win=65535 Len=0
14	0.000000	192.168.163.103	192.168.163.10	HTTP	141 GET / HTTP/1.1
15	0.000000	192.168.163.10	192.168.163.103	TCP	60 80 → 55428 [ACK] Seq=1 Ack=88 Win=65535 Len=0

Wireshark 打开 out.pcap 文件:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	ARP Announcement for 192.168.163.103
2	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	Who has 192.168.163.10? Tell 192.168.163.103
3	0.000000	192.168.163.103	192.168.163.10	DNS	84	Standard query 0x96da A www.baidu.com OPT
4	0.000000	192.168.163.103	192.168.163.10	DNS	84	Standard query 0x9759 AAAA www.baidu.com OPT
5	0.000000	192.168.163.103	192.168.163.10	DNS	87	Standard query 0x5a54 AAAA www.a.shifen.com OPT
6	0.000000	11:22:33:44:55:66	1a:94:f0:3c:49:aa	ARP	60	192.168.163.103 is at 11:22:33:44:55:66
7	0.000000	192.168.163.103	192.168.163.110	ICMP	98	Echo (ping) reply id=0x0001, seq=1/256, ttl=64
8	0.000000	192.168.163.103	192.168.163.2	TCP	60	22 → 64289 [ACK] Seq=1 Ack=53 Win=65535 Len=0
9	0.000000	192.168.163.103	192.168.163.10	HTTP	141	GET / HTTP/1.1

Wireshark 打开 demo_out.pcap 文件:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	ARP Announcement for 192.168.163.103
2	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	Who has 192.168.163.10? Tell 192.168.163.103
3	0.000000	192.168.163.103	192.168.163.10	DNS	84	Standard query 0x96da A www.baidu.com OPT
4	0.000000	192.168.163.103	192.168.163.10	DNS	84	Standard query 0x9759 AAAA www.baidu.com OPT
5	0.000000	192.168.163.103	192.168.163.10	DNS	87	Standard query 0x5a54 AAAA www.a.shifen.com OPT
6	0.000000	11:22:33:44:55:66	1a:94:f0:3c:49:aa	ARP	60	192.168.163.103 is at 11:22:33:44:55:66
7	0.000000	192.168.163.103	192.168.163.110	ICMP	98	Echo (ping) reply id=0x0001, seq=1/256, ttl=64
8	0.000000	192.168.163.103	192.168.163.2	TCP	60	22 → 64289 [ACK] Seq=1 Ack=1 Win=65535 Len=0
9	0.000000	192.168.163.103	192.168.163.10	HTTP	141	GET / HTTP/1.1

对比可知，demo_out 与 out 完全相同，选取 out 中的第 9 个报文，即 HTTP 请求进行分析：

```

> Frame 9: 141 bytes on wire (1128 bits), 141 bytes captured (1128 bits)
> Ethernet II, Src: 11:22:33:44:55:66 (11:22:33:44:55:66), Dst: 21:32:43:54:65:06 (21:32:43:54:65:06)
> Destination: 21:32:43:54:65:06 (21:32:43:54:65:06)
> Source: 11:22:33:44:55:66 (11:22:33:44:55:66)
Type: IPv4 (0x0800)
Stream index: 1
> Internet Protocol Version 4, Src: 192.168.163.103, Dst: 192.168.163.10
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 127
Identification: 0x0005 (5)
0000 .... = Flags: 0x0
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 64
Protocol: TCP (6)
Header Checksum: 0xb2b1 [validation disabled]
[Header checksum status: Unverified]
Source Address: 192.168.163.103
Destination Address: 192.168.163.10
[Stream index: 0]
> Transmission Control Protocol, Src Port: 55428, Dst Port: 80, Seq: 1, Ack: 1, Len: 87
> Hypertext Transfer Protocol

```

由图可知，报文的 IP 头部信息：

版本: 4

首部长度: 20 字节 (5)

总长度: 127

标识: 0x0005 (5)

标志: 0x00

片偏移: 0

生存时间: 64

协议: TCP (6)

首部校验和: 0xb2b1

源 IP 地址: 192.168.163.103

目的 IP 地址: 192.168.163.10

上述信息与 demo_out 中的该条报文的 IP 头部信息相同，表明 IP 头部填充、校验和计算均正确，IP 协议相关函数实现正确。其余报文分析类似。

ip_frag_test 测试并没有给出输入输出的 pcap 文件，该测试使用一个很长的 in.txt 文件作为输入，将分片发送结果输出在 log 文件中，上述 log 文件对比中已经说明与 demo_log 的一致性，所以 IP 分片相关的操作也是正确实现的。

4. ICMP 协议实验结果及分析

(展示 ICMP 报文的捕获截图，分析其报文内容（包括差错报文和查询报文）。检查 ICMP 类型、代码、校验和等字段，以及报文携带的信息。)

能够通过 icmp_test 测试:

```
(base) PS C:\Users\Jin_zd\Desktop\Code\Protocol-Stack-Lab\build> ctest -R icmp_test
Test project C:/Users/Jin_zd/Desktop/Code/Protocol-Stack-Lab/build
Start 6: icmp_test
1/1 Test #6: icmp_test ..... Passed    0.05 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) = 0.05 sec
```

demo_log 和 log 文件对比没有不同:

```
1 driver opened
2 <===== arp table =====>
3 <===== arp buf =====>
4
5 Round 01 -----
6 <===== arp table =====>
7 <===== arp buf =====>
8 192.168.163.10 -> 45 00 00 46 00 00 00 40 11 b2 e4 c0 a8 a3 67 c0 a8 a3 0a
9
10 Round 02 -----
11 <===== arp table =====>
12 192.168.163.10 -> 21:32:43:54:65:06
13 <===== arp buf =====>
14
15 Round 03 -----
16 <===== arp table =====>
17 192.168.163.10 -> 21:32:43:54:65:06
18 <===== arp buf =====>
19
20 Round 04 -----
21 <===== arp table =====>
22 192.168.163.10 -> 21:32:43:54:65:06
23 <===== arp buf =====>
24
25 Round 05 -----
26 udp_in:
27   src_ip:192.168.163.10
28   buf: 00 35 ae 1b 00 6d bb f0 96 da 81 80 00 01 00 03 00 00 00 01 03 77 77 77
29 <===== arp table =====>
30 192.168.163.10 -> 21:32:43:54:65:06
31 <===== arp buf =====>
32
33 Round 06 -----
34 <===== arp table =====>
35 192.168.163.10 -> 21:32:43:54:65:06
36 <===== arp buf =====>
37
38 Round 07 -----
39 udp_in:
40   src_ip:192.168.163.10
41   buf: 00 35 84 9f 80 74 72 81 5a 54 81 80 00 01 00 00 00 01 00 01 03 77 77 81
42 <===== arp table =====>
43 192.168.163.10 -> 21:32:43:54:65:06
```

Wireshark 打开 in.pcap 文件:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.163.103	192.168.163.10	DNS	84	Standard query 0x96da A www.baidu.com OPT
2	0.000000	21:32:43:54:65:06	11:22:33:44:55:66	ARP	42	192.168.163.10 is at 21:32:43:54:65:06
3	0.000000	192.168.163.103	192.168.163.10	DNS	84	Standard query response 0x9759 AAAA www.baidu.com OPT
4	0.000000	192.168.163.103	192.168.163.103	DNS	168	Standard query response 0x9759 AAAA www.baidu.com CNAME www.a.shifen.com SOA ns1.a.shifen.com OPT
5	0.000000	192.168.163.10	192.168.163.103	DNS	143	Standard query response 0x96da A www.baidu.com CNAME www.a.shifen.com A 183.232.231.172 OPT
6	0.000000	192.168.163.103	192.168.163.10	DNS	87	Standard query 0x5a54 AAAA www.a.shifen.com OPT
7	0.000000	192.168.163.10	192.168.163.103	DNS	150	Standard query response 0x5a54 AAAA www.a.shifen.com SOA ns1.a.shifen.com OPT
8	0.000000	192.168.163.110	192.168.163.103	ICMP	98	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 11)
9	0.000000	Pixin_34:45:56	11:22:33:44:55:66	ARP	42	192.168.163.110 is at 01:12:23:34:45:56
10	0.000000	1a:94:f0:3c:49:aa	Broadcast	ARP	42	Who has 192.168.163.103? Tell 192.168.163.2
11	0.000000	192.168.163.103	192.168.163.110	ICMP	98	Echo (ping) reply id=0x0001, seq=1/256, ttl=64
12	0.000000	192.168.163.2	192.168.163.103	SSH	106	Client: Encrypted packet (len=52)
13	0.000000	192.168.163.103	192.168.163.2	TCP	54	22 → 64289 [ACK] Seq=1 Ack=53 Win=65535 Len=0
14	0.000000	192.168.163.103	192.168.163.10	HTTP	141	GET / HTTP/1.1
15	0.000000	192.168.163.10	192.168.163.103	TCP	60	80 → 55428 [ACK] Seq=1 Ack=88 Win=65535 Len=0

Wireshark 打开 out.pcap 文件:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	ARP Announcement for 192.168.163.103
2	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	Who has 192.168.163.10? Tell 192.168.163.103
3	0.000000	192.168.163.103	192.168.163.10	DNS	84	Standard query 0x96da A www.baidu.com OPT
4	0.000000	192.168.163.103	192.168.163.10	DNS	84	Standard query 0x9759 AAAA www.baidu.com OPT
5	0.000000	192.168.163.103	192.168.163.10	DNS	87	Standard query 0x5a54 AAAA www.a.shifen.com OPT
6	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	Who has 192.168.163.110? Tell 192.168.163.103
7	0.000000	192.168.163.103	192.168.163.110	ICMP	98	Echo (ping) reply id=0x0001, seq=1/256, ttl=64
8	0.000000	11:22:33:44:55:66	1a:94:f0:3c:49:aa	ARP	60	192.168.163.103 is at 11:22:33:44:55:66
9	0.000000	192.168.163.103	192.168.163.110	ICMP	98	Echo (ping) reply id=0x0001, seq=1/256, ttl=64
10	0.000000	192.168.163.103	192.168.163.2	ICMP	70	Destination unreachable (Protocol unreachable)
11	0.000000	192.168.163.103	192.168.163.2	TCP	60	22 → 64289 [ACK] Seq=1 Ack=1 Win=65535 Len=0
12	0.000000	192.168.163.103	192.168.163.10	HTTP	141	GET / HTTP/1.1
13	0.000000	192.168.163.103	192.168.163.10	ICMP	70	Destination unreachable (Protocol unreachable)

Wireshark 打开 demo_out.pcap 文件:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	ARP Announcement for 192.168.163.103
2	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	Who has 192.168.163.10? Tell 192.168.163.103
3	0.000000	192.168.163.103	192.168.163.10	DNS	84	Standard query 0x96da A www.baidu.com OPT
4	0.000000	192.168.163.103	192.168.163.10	DNS	84	Standard query 0x9759 AAAA www.baidu.com OPT
5	0.000000	192.168.163.103	192.168.163.10	DNS	87	Standard query 0x5a54 AAAA www.a.shifen.com OPT
6	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	Who has 192.168.163.110? Tell 192.168.163.103
7	0.000000	192.168.163.103	192.168.163.110	ICMP	98	Echo (ping) reply id=0x0001, seq=1/256, ttl=64
8	0.000000	11:22:33:44:55:66	1a:94:f0:3c:49:aa	ARP	60	192.168.163.103 is at 11:22:33:44:55:66
9	0.000000	192.168.163.103	192.168.163.110	ICMP	98	Echo (ping) reply id=0x0001, seq=1/256, ttl=64
10	0.000000	192.168.163.103	192.168.163.2	ICMP	70	Destination unreachable (Protocol unreachable)
11	0.000000	192.168.163.103	192.168.163.2	TCP	60	22 → 64289 [ACK] Seq=1 Ack=1 Win=65535 Len=0
12	0.000000	192.168.163.103	192.168.163.10	HTTP	141	GET / HTTP/1.1
13	0.000000	192.168.163.103	192.168.163.10	ICMP	70	Destination unreachable (Protocol unreachable)

demo_out 和 out 文件相同。

in 中的第 8 条报文成功收到了一个 ICMP 请求，第 11 条报文成功收到了一个 ICMP 应答，out 中的第 7 条报文和第 9 条报文均成功发送了 ICMP 应答。out 中的第 10 条和第 13 条报文均成功发送了 ICMP 不可达报文。

选取 out 中的第 7 条报文进行分析：

> Frame 7: 98 bytes on wire (784 bits), 98 bytes captured (784 bits)

> Ethernet II, Src: 11:22:33:44:55:66 (11:22:33:44:55:66), Dst: Ppim_34:45:56 (01:12:23:34:45:56)

> Internet Protocol Version 4, Src: 192.168.163.103, Dst: 192.168.163.110

> Internet Control Message Protocol

Type: 0 (Echo (ping) reply)

Code: 0

Checksum: 0x436a [correct]

[Checksum Status: Good]

Identifier (BE): 1 (0x0001)

Identifier (LE): 256 (0x0100)

Sequence Number (BE): 1 (0x0001)

Sequence Number (LE): 256 (0x0100)

> Data (56 bytes)

Data: c8e4865f00000000a7c00000000000101112131415161718191a1b1c1d1e1f202122232425262728292a2b2c2d...

[Length: 56]

0000 01 12 23 34 45 56 11 22 33 44 55 66 08 00 45 00 ..#4EV-"3DUf--E-

0010 00 54 00 03 00 00 40 01 b2 7f c0 a8 a3 67 c0 a8 -T---@---g-

0020 a3 6a 80 00 43 6a 00 01 00 01 c8 e4 86 5f 00 00 -n-CJ---.....

0030 00 00 a0 7c 00 00 00 00 00 00 11 12 13 14 15 ..|---.....

0040 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 251*5\$

0050 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 &(')*+.../012345

0060 36 37

由图可知，报文的 ICMP 头部信息：
类型: 0 (Echo (ping) reply - 回显应答)
代码: 0
校验和: 0x436a

同样，选取 out 的第 10 条报文进行分析：
由图可知，报文的 ICMP 头部信息：

> Frame 10: 70 bytes on wire (560 bits), 70 bytes captured (560 bits)

> Ethernet II, Src: 11:22:33:44:55:66 (11:22:33:44:55:66), Dst: 1a:94:f0:3c:49:aa (1a:94:f0:3c:49:aa)

> Internet Protocol Version 4, Src: 192.168.163.103, Dst: 192.168.163.2

> Internet Control Message Protocol

Type: 3 (Destination unreachable)

Code: 2 (Protocol unreachable)

Checksum: 0xe5eb [correct]

[Checksum Status: Good]

Unused: 00000000

> Internet Protocol Version 4, Src: 192.168.163.2, Dst: 192.168.163.103

> Transmission Control Protocol, Src Port: 64289, Dst Port: 22

0000 1a 94 f0 3c 49 aa 11 22 33 44 55 66 08 00 45 00 ...-I- "3DUf--E-

0010 00 38 00 05 00 00 40 01 b3 05 c0 a8 a3 67 c0 a8 -8---@---g-

0020 a3 92 10 02 a3 a3 00 00 00 00 50 00 03 5c 2f 00 -.-@---E-V-

0030 00 00 40 06 29 f7 c0 a8 a3 92 c0 a8 a3 67 fb 21 -@---.....g-

0040 00 16 22 ea f8 ef

类型: 3 (Destination unreachable - 目的主机不可达)
代码: 2 (Protocol unreachable - 协议不可达)
校验和: 0xe5eb

上述两个报文的分析表明 ICMP 的回显报文和差错报文均实现正确，其余报文的分析类似，表明 ICMP 协议相关函数的实现正确。

5. UDP 协议实验结果及分析

(展示 UDP 数据包的捕获截图，解析 UDP 头部和载荷内容，分析是否达到预期。检查源端口号、目的端口号、长度、校验和等字段，以及载荷数据。)

能够通过 udp_test 测试:

```
(base) PS C:\Users\Jin_zd\Desktop\Code\Protocol-Stack-Lab\build> ctest -R udp_test
Test project C:/Users/Jin_zd/Desktop/Code/Protocol-Stack-Lab/build
Start 7: udp_test
1/1 Test #7: udp_test ..... Passed    0.05 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) =  0.05 sec
```

demo_log 和 log 文件对比, 没有不同:

```
Testing 2 data 2 udp_test 2 log
1 driver opened
2 <===== arp table =====>
3 <===== arp buf =====>
4
5 Round 01 -----
6 <===== arp table =====>
7 192.168.163.10 -> 21:32:43:54:65:06
8 <===== arp buf =====>
9
10 Round 02 -----
11 <===== arp table =====>
12 192.168.163.10 -> 21:32:43:54:65:06
13 <===== arp buf =====>
14
15 Round 03 -----
16 <===== arp table =====>
17 192.168.163.10 -> 21:32:43:54:65:06
18 <===== arp buf =====>
19
20 Round 04 -----
21 <===== arp table =====>
22 192.168.163.10 -> 21:32:43:54:65:06
23 <===== arp buf =====>
24
25 driver closed
26
```

Wireshark 打开 in.pcap 文件:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	21:32:43:54:65:06	11:22:33:44:55:66	ARP	42	192.168.163.10 is at 21:32:43:54:65:06
2	-882.530372	192.168.163.10	192.168.163.103	UDP	47	59040 → 60000 Len=5
3	-880.309687	192.168.163.10	192.168.163.103	UDP	47	59040 → 60000 Len=5
4	-878.779374	192.168.163.10	192.168.163.103	UDP	47	59040 → 60000 Len=5

Wireshark 打开 out.pcap 文件:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	ARP Announcement for 192.168.163.103
2	0.000000	192.168.163.103	192.168.163.10	UDP	60	60000 → 59040 Len=5
3	0.000000	192.168.163.103	192.168.163.10	UDP	60	60000 → 59040 Len=5
4	0.000000	192.168.163.103	192.168.163.10	UDP	60	60000 → 59040 Len=5

Wireshark 打开 demo_out.pcap 文件:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	ARP Announcement for 192.168.163.103
2	50.062120	192.168.163.103	192.168.163.10	UDP	60	60000 → 59040 Len=5
3	52.273372	192.168.163.103	192.168.163.10	UDP	60	60000 → 59040 Len=5
4	53.802411	192.168.163.103	192.168.163.10	UDP	60	60000 → 59040 Len=5

demo_out 与 out 文件相同。

out 中的第 2、3、4 条报文白表明发送了 UDP 数据报, in 中的第 2、3、4 条报文表明成功接收到了这几条报文。

选取 out 中的第 2 条 UDP 报文进行分析:

```
> Frame 2: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface 0
> Ethernet II, Src: 11:22:33:44:55:66 (11:22:33:44:55:66), Dst: 21:32:43:54:65:06 (21:32:43:54:65:06)
> Internet Protocol Version 4, Src: 192.168.163.103, Dst: 192.168.163.10
  User Datagram Protocol, Src Port: 60000, Dst Port: 59040
    Source Port: 60000
    Destination Port: 59040
    Length: 13
    Checksum: 0xd34c [unverified]
    [Checksum Status: Unverified]
    [Stream index: 0]
    [Stream Packet Number: 1]
  > [Timestamps]
    UDP payload (5 bytes)
  > Data (5 bytes)
```

由图可知，该报文的 UDP 头部信息：

源端口: 60000

目的端口: 59040

长度: 13

校验和: 0x434c

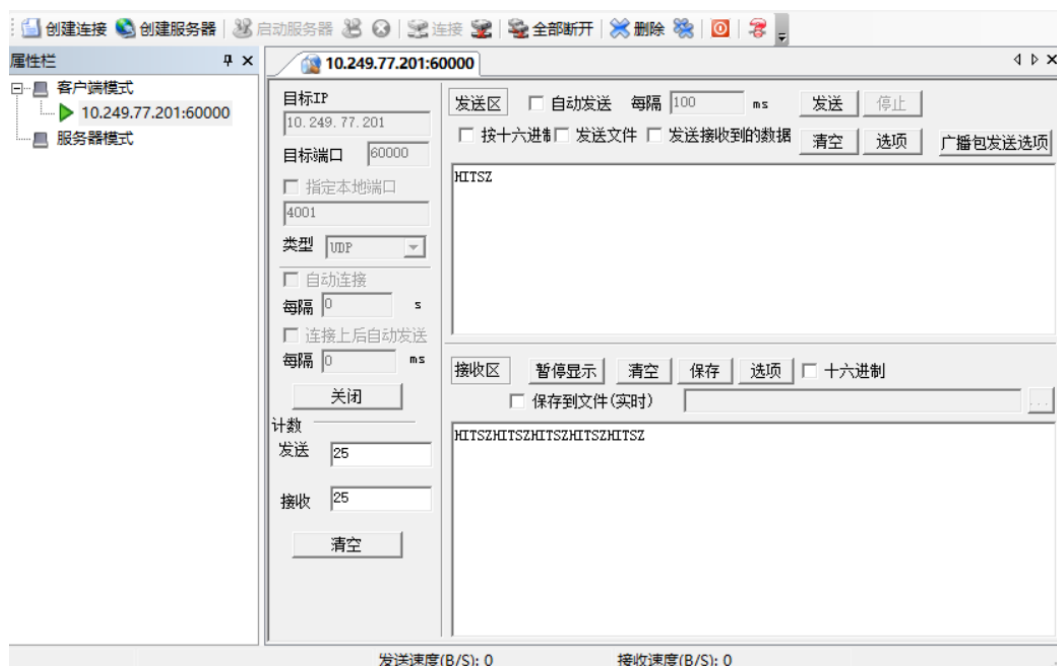
载荷数据: 5 字节

UDP 头部相关信息均正确，其余报文分析类似。所以 UDP 协议相关函数实现正确。

对于 udp_sever 测试，在终端运行 main 函数：

```
(d1) PS C:\Users\Jin_zd\Desktop\Code\Protocol-Stack-Lab\build> . "C:/Users/Jin_zd/Desktop/Code/Protocol-Stack-Lab/build/udp_server.exe"
Using interface \Device\NPF_{6F6D7EFA-327D-4091-A9A4-6F6228C84BDF}, my ip is 10.249.77.201.
```

随后，使用 TCP&UDP 测试工具，在 10.249.77.201:60000 上建立 UDP 连接，发送五次 HITSZ 的报文：



终端可以正确收到这五条报文：

```
(d1) PS C:\Users\Jin_zd\Desktop\Code\Protocol-Stack-Lab\build> . "C:/Users/Jin_zd/Desktop/Code/Protocol-Stack-Lab/build/udp_server.exe"
Using interface \Device\NPF_{6F6D7EFA-327D-4091-A9A4-6F6228C84BDF}, my ip is 10.249.77.201.
recv udp packet from 10.249.77.200:49977 len=5
HITSZ
arp_out: 10.249.77.200 not in arp_buf
recv udp packet from 10.249.77.200:49977 len=5
HITSZ
recv udp packet from 10.249.77.200:49977 len=5
HITSZ
recv udp packet from 10.249.77.200:49977 len=5
HITSZ
recv udp packet from 10.249.77.200:49977 len=5
HITSZ
recv udp packet from 10.249.77.200:49977 len=5
HITSZ
```

使用 Wireshark 在筛选 eth.addr==11:22:33:44:55:66（即 10.249.77.201 的 MAC 地址）条件下的报文：

eth.addr=11:22:33:44:55:66						
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	ARP Announcement for 10.249.77.201
2	8.960880	10.249.77.200	10.249.77.201	UDP	47	49977 → 60000 Len=5
3	8.962368	11:22:33:44:55:66	Broadcast	ARP	60	Who has 10.249.77.200? Tell 10.249.77.201
4	8.962413	Microsoft_4d:66:01	11:22:33:44:55:66	ARP	42	10.249.77.200 is at 00:15:5d:4d:66:01
5	8.963229	10.249.77.201	10.249.77.200	UDP	60	60000 → 49977 Len=5
6	9.345579	10.249.77.200	10.249.77.201	UDP	47	49977 → 60000 Len=5
7	9.346196	10.249.77.201	10.249.77.200	UDP	60	60000 → 49977 Len=5
8	9.729891	10.249.77.200	10.249.77.201	UDP	47	49977 → 60000 Len=5
9	9.730476	10.249.77.201	10.249.77.200	UDP	60	60000 → 49977 Len=5
10	10.136198	10.249.77.200	10.249.77.201	UDP	47	49977 → 60000 Len=5
11	10.136474	10.249.77.201	10.249.77.200	UDP	60	60000 → 49977 Len=5
12	10.587148	10.249.77.200	10.249.77.201	UDP	47	49977 → 60000 Len=5
13	10.587450	10.249.77.201	10.249.77.200	UDP	60	60000 → 49977 Len=5

可以看到，正确进行了五轮 UDP 通信，即从 49977 端口到 60000 端口的发送报文和从 60000 端口到 49977 的回复一共是五轮，选取第 12 条报文进行分析：

> Frame 12: 47 bytes on wire (376 bits), 47 bytes captured (376 bits) on interface \Device\NPF_{6F6D7EFA-3}

> Ethernet II, Src: Microsoft_4d:66:01 (00:15:5d:4d:66:01), Dst: 11:22:33:44:55:66 (11:22:33:44:55:66)

> Internet Protocol Version 4, Src: 10.249.77.200, Dst: 10.249.77.201

> User Datagram Protocol, Src Port: 49977, Dst Port: 60000

Source Port: 49977

Destination Port: 60000

Length: 13

Checksum: 0xaa19 [unverified]

[Checksum Status: Unverified]

[Stream index: 0]

[Stream Packet Number: 9]

> [Timestamps]

> UDP payload (5 bytes)

> Data (5 bytes)

Data: 4849454535a

[Length: 5]

0000 11 22 33 44 55 66 00 15 5d 4d 66 01 08 00 45 00 - "3DUF...]HF...E-

0010 00 21 ab 4c 00 00 00 11 dd fc 0a f9 4d c8 0a f9 - .L.....M...

0020 4d c9 39 ea 60 00 00 aa 19 48 49 54 53 5a M 9... HITSZ

由图可知，报文的 UDP 信息为：

源端口: 49977

目的端口: 60000

长度: 13

校验和: 0xaa19

载荷数据: 5 字节

数据: 4849454535a (HITSZ)

长度: 5

UDP 头部信息和数据信息正确，说明信息 UDP 头部的填充正确，数据信息的传输正确，表明 UDP 协议相关函数实现正确。能够正确的接受和发送 UDP 报文，还表明代码的以太网协议、ARP 协议、IP 协议、ICMP 协议相关函数均实现正确。

6. TCP 协议实验结果及分析

(展示 TCP 数据包的捕获截图，分析 TCP 连接的建立、数据传输和关闭过程。检查 TCP 头部的源端口号、目的端口号、序列号、确认号、标志位等字段，以及连接的状态转换。)

能够通过 tcp_test 测试：

```
(base) PS C:\Users\Jin_zd\Desktop\Code\Protocol-Stack-Lab\build> ctest -R tcp_test
Test project C:/Users/Jin_zd/Desktop/Code/Protocol-Stack-Lab/build
  Start 8: tcp_test
1/1 Test #8: tcp_test ..... Passed    0.05 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) = 0.05 sec
```

demo_log 和 log 文件对比，没有不同：

```

1 driver opened
2 <===== arp table =====
3 <===== arp buf =====
4
5 Round 01 -----
6 <===== arp table =====
7 192.168.163.10 -> 21:32:43:54:65:06
8 <===== arp buf =====
9
10 Round 02 -----
11 <===== arp table =====
12 192.168.163.10 -> 21:32:43:54:65:06
13 <===== arp buf =====
14
15 Round 03 -----
16 <===== arp table =====
17 192.168.163.10 -> 21:32:43:54:65:06
18 <===== arp buf =====
19
20 Round 04 -----
21 <===== arp table =====
22 192.168.163.10 -> 21:32:43:54:65:06
23 <===== arp buf =====
24
25 Round 05 -----
26 <===== arp table =====
27 192.168.163.10 -> 21:32:43:54:65:06
28 <===== arp buf =====
29
30 Round 06 -----
31 <===== arp table =====
32 192.168.163.10 -> 21:32:43:54:65:06
33 <===== arp buf =====
34
35 Round 07 -----
36 <===== arp table =====
37 192.168.163.10 -> 21:32:43:54:65:06
38 <===== arp buf =====
39
40 Round 08 -----
41 <===== arp table =====
42 192.168.163.10 -> 21:32:43:54:65:06
43 <===== arp buf =====

```

Wireshark 打开 in.pcap 文件：

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	21:32:43:54:65:06	11:22:33:44:55:66	ARP	42	192.168.163.10 is at 21:32:43:54:65:06
2	-5696.528861	192.168.163.10	192.168.163.103	TCP	66	56446 → 60000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
3	-5696.517291	192.168.163.10	192.168.163.103	TCP	54	56446 → 60000 [ACK] Seq=1 Ack=1 Win=16740352 Len=0
4	-5692.255047	192.168.163.10	192.168.163.103	TCP	55	56446 → 60000 [PSH, ACK] Seq=1 Ack=1 Win=16740352 Len=1
5	-5692.203932	192.168.163.10	192.168.163.103	TCP	54	56446 → 60000 [ACK] Seq=2 Ack=2 Win=16740096 Len=0
6	-5689.624319	192.168.163.10	192.168.163.103	TCP	55	56446 → 60000 [PSH, ACK] Seq=2 Ack=2 Win=16740096 Len=1
7	-5689.573523	192.168.163.10	192.168.163.103	TCP	54	56446 → 60000 [ACK] Seq=3 Ack=3 Win=16739840 Len=0
8	-5687.985512	192.168.163.10	192.168.163.103	TCP	55	56446 → 60000 [PSH, ACK] Seq=3 Ack=3 Win=16739840 Len=1
9	-5687.928633	192.168.163.10	192.168.163.103	TCP	54	56446 → 60000 [ACK] Seq=4 Ack=4 Win=16739584 Len=0
10	-5685.820059	192.168.163.10	192.168.163.103	TCP	54	56446 → 60000 [FIN, ACK] Seq=4 Ack=4 Win=16739584 Len=0
11	-5685.811252	192.168.163.10	192.168.163.103	TCP	54	56446 → 60000 [ACK] Seq=5 Ack=5 Win=16739584 Len=0

Wireshark 打开 out.pcap 文件：

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	ARP Announcement for 192.168.163.103
2	0.000000	192.168.163.103	192.168.163.10	TCP	60	60000 → 56446 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0
3	0.000000	192.168.163.103	192.168.163.10	TCP	60	60000 → 56446 [ACK] Seq=1 Ack=2 Win=65535 Len=1
4	0.000000	192.168.163.103	192.168.163.10	TCP	60	60000 → 56446 [ACK] Seq=2 Ack=3 Win=65535 Len=1
5	0.000000	192.168.163.103	192.168.163.10	TCP	60	60000 → 56446 [ACK] Seq=3 Ack=4 Win=65535 Len=1
6	0.000000	192.168.163.103	192.168.163.10	TCP	60	60000 → 56446 [FIN, ACK] Seq=4 Ack=5 Win=65535 Len=0

Wireshark 打开 demo_out.pcap 文件：

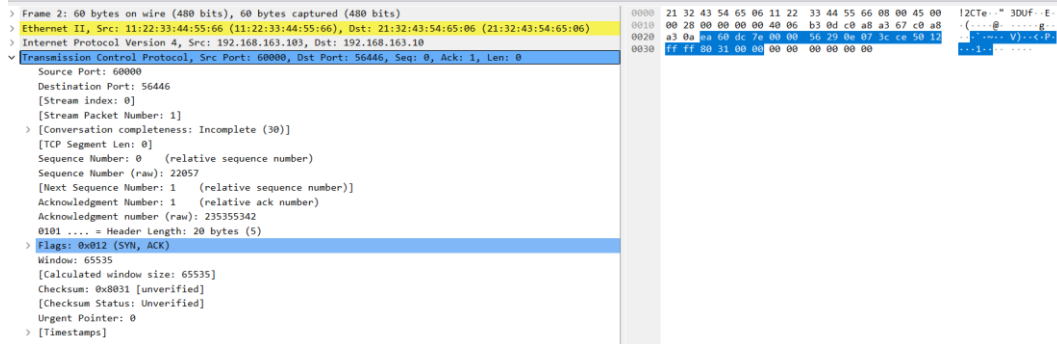
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	ARP Announcement for 192.168.163.103
2	15.918779	192.168.163.103	192.168.163.10	TCP	60	60000 → 56446 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0
3	20.186985	192.168.163.103	192.168.163.10	TCP	60	60000 → 56446 [ACK] Seq=1 Ack=2 Win=65535 Len=1
4	22.816084	192.168.163.103	192.168.163.10	TCP	60	60000 → 56446 [ACK] Seq=2 Ack=3 Win=65535 Len=1
5	24.455263	192.168.163.103	192.168.163.10	TCP	60	60000 → 56446 [ACK] Seq=3 Ack=4 Win=65535 Len=1
6	26.624398	192.168.163.103	192.168.163.10	TCP	60	60000 → 56446 [FIN, ACK] Seq=4 Ack=5 Win=65535 Len=0

Demo_out 和 out 文件相同。

in 中的第 2 条报文表明收到了 TCP 连接请求，其 SYN=1，out 中的第 2 条报文表明发出来 TCP 连接请求确认，其 SYN=1，ACK=1，in 中的第 3 条报文表明对该确认报文的确认，其 ACK=1，至此，TCP 连接建立的三次握手结束。

in 中的第 10 条报文表明收到了 TCP 终止连接请求，其 FIN=1，out 中的第 6 条报文表明数据传输也已完成，并请求关闭连接，其 FIN=1，ACK=1，最后，in 中的第 11 条报文表明收到 FIN 后，发送 ACK 进行确认，至此，TCP 连接关闭的四次挥手完成。

选取 out 的第 2 条报文进行分析：



由图可知，报文的 TCP 头部信息：

源端口: 60000

目的端口: 56446

序列号: 0 (相对序列号)

序列号 (原始): 22057

下一个序列号: 1 (相对序列号)

确认号: 1 (相对确认号)

确认号 (原始): 235355342

首部长: 20 字节 (5)

标志: 0x012 (SYN, ACK)

窗口: 65535

计算窗口大小: 65535

校验和: 0x0b31

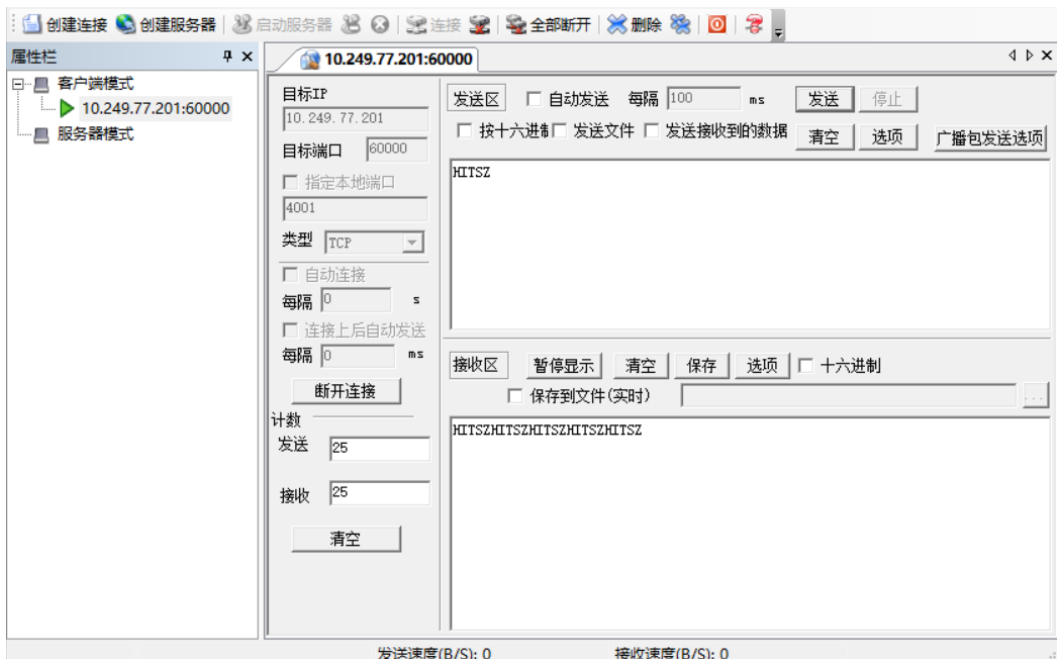
紧急指针: 0

符合 TCP 请求确认报文的信息要求，其余报文分析类似。所以 TCP 协议相关函数实现正确。

对于 tcp_server 测试，在终端中运行 main:

```
(d1) PS C:\Users\Jin_zd\Desktop\Code\Protocol-Stack-Lab\build> . "C:/Users/Jin_zd/Desktop/Code/Protocol-Stack-Lab/build/tcp_server.exe"
Using interface \Device\NPF_{6F6D7EFA-327D-4091-A9A4-6F6228C84BDF}, my ip is 10.249.77.201.
```

随后，使用 TCP&UDP 测试工具，在 10.249.77.201:60000 上建立 TCP 连接，发送五次 HITSZ 的报文：



终端可以正确收到这五条报文：

```
(d1) PS C:\Users\jin_zd\Desktop\Code\Protocol-Stack-Lab\build> . "C:\Users\jin_zd\Desktop\Code\Protocol-Stack-Lab\build\tcp_server.exe"
Using interface \Device\NPF_{6F607EFA-327D-4091-A9A4-6F6228C84B0F}, my ip is 10.249.77.201.
arp_out: 10.249.77.200 not in arp_buf
HITSZ
HITSZ
HITSZ
HITSZ
HITSZ
```

使用 Wireshark 在筛选 eth.addr==11:22:33:44:55:66（即 10.249.77.201 的 MAC 地址）条件下的报文：

eth.addr==11:22:33:44:55:66							
No.	Time	Source	Destination	Protocol	Length	Info	
1	0.000000	11:22:33:44:55:66	Broadcast	ARP	60	ARP Announcement for 10.249.77.201	
2	5.442741	10.249.77.200	10.249.77.201	TCP	66	13785 → 60000 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM	
3	5.444597	11:22:33:44:55:66	Broadcast	ARP	60	Who has 10.249.77.200? Tell 10.249.77.201	
4	5.444640	Microsoft_D4:66:01	11:22:33:44:55:66	ARP	42	10.249.77.200 is at 00:15:5d:4d:66:01	
5	5.445543	10.249.77.201	10.249.77.200	TCP	60	60000 → 13785 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0	
6	5.445668	10.249.77.200	10.249.77.201	TCP	54	13785 → 60000 [ACK] Seq=1 Ack=1 Win=65535 Len=0	
7	7.233372	10.249.77.200	10.249.77.201	TCP	59	13785 → 60000 [PSH, ACK] Seq=1 Ack=1 Win=65535 Len=5	
8	7.233796	10.249.77.201	10.249.77.200	TCP	60	60000 → 13785 [ACK] Seq=1 Ack=6 Win=65535 Len=0	
9	7.286696	10.249.77.200	10.249.77.201	TCP	54	13785 → 60000 [ACK] Seq=6 Ack=6 Win=65530 Len=0	
10	7.596113	10.249.77.200	10.249.77.201	TCP	59	13785 → 60000 [PSH, ACK] Seq=6 Ack=6 Win=65530 Len=5	
11	7.596418	10.249.77.201	10.249.77.200	TCP	60	60000 → 13785 [ACK] Seq=6 Ack=11 Win=65535 Len=5	
12	7.646604	10.249.77.200	10.249.77.201	TCP	54	13785 → 60000 [ACK] Seq=11 Ack=11 Win=65525 Len=0	
13	7.960350	10.249.77.200	10.249.77.201	TCP	59	13785 → 60000 [PSH, ACK] Seq=11 Ack=11 Win=65525 Len=5	
14	7.961109	10.249.77.201	10.249.77.200	TCP	60	60000 → 13785 [ACK] Seq=11 Ack=16 Win=65535 Len=5	
15	8.006343	10.249.77.200	10.249.77.201	TCP	54	13785 → 60000 [ACK] Seq=16 Ack=16 Win=65520 Len=0	
16	8.344397	10.249.77.200	10.249.77.201	TCP	59	13785 → 60000 [PSH, ACK] Seq=16 Ack=16 Win=65520 Len=5	
17	8.344655	10.249.77.201	10.249.77.200	TCP	60	60000 → 13785 [ACK] Seq=16 Ack=21 Win=65535 Len=5	
18	8.397143	10.249.77.200	10.249.77.201	TCP	54	13785 → 60000 [ACK] Seq=21 Ack=21 Win=65515 Len=0	
19	8.709645	10.249.77.200	10.249.77.201	TCP	59	13785 → 60000 [PSH, ACK] Seq=21 Ack=21 Win=65515 Len=5	
20	8.710302	10.249.77.201	10.249.77.200	TCP	60	60000 → 13785 [ACK] Seq=21 Ack=26 Win=65535 Len=5	
21	8.753957	10.249.77.200	10.249.77.201	TCP	54	13785 → 60000 [ACK] Seq=26 Ack=26 Win=65510 Len=0	

分析图像，可知：

TCP 连接建立 (三次握手)：

2 号数据报，10.249.77.200:14343 向 10.249.77.201:60000 发送，SYN=1, Seq=0 表明请求建立 TCP 连接。

5 号数据报，10.249.77.201:60000 向 10.249.77.200:14343 发送，SYN=1, ACK=1, Seq=0, Ack=1 表明同意建立连接并确认客户端 SYN。

6 号数据报，10.249.77.200:14343 向 10.249.77.201:60000 发送，ACK=1, Seq=1, Ack=1 表明确认服务器 SYN+ACK，TCP 连接建立成功。

TCP 数据传输:

7 号数据报, 10.249.77.200:14343 向 10.249.77.201:60000 发送, PSH=1, ACK=1, Seq=1, Ack=1, Len=5 表明客户端开始向服务器发送数据 (5 字节)。

8 号数据报, 10.249.77.201:60000 向 10.249.77.200:14343 发送, ACK=1, Seq=1, Ack=6 表明服务器确认收到客户端数据, 期望下一个序列号为 6。

10 号数据报, 10.249.77.200:14343 向 10.249.77.201:60000 发送, PSH=1, ACK=1, Seq=6, Ack=6, Len=5 表明客户端继续发送数据 (5 字节)。

接下来的数据报 (9、11-21 数据报), 10.249.77.200:14343 向 10.249.77.201:60000 发送或接收, 主要为 PSH/ACK 或 ACK, 表明客户端和服务器之间持续进行数据传输和确认。

TCP 连接释放 (四次挥手):

22 号数据报, 10.249.77.200:14343 向 10.249.77.201:60000 发送, FIN=1, ACK=1, Seq=26, Ack=26 表明客户端请求关闭连接。

23 号数据报, 10.249.77.201:60000 向 10.249.77.200:14343 发送, FIN=1, ACK=1, Seq=26, Ack=27 表明服务器确认收到客户端 FIN 并请求关闭连接。

24 号数据报, 10.249.77.200:14343 向 10.249.77.201:60000 发送, ACK=1, Seq=27, Ack=27 表明客户端确认收到服务器 FIN, TCP 连接完全关闭。

根据上述分析, 服务器和客户端之间可以完成完整的 TCP 连接建立、数据传输、连接释放过程, 表明 TCP 协议相关函数实现正确, 同时表明 TCP 以下各个协议相关函数的实现正确。

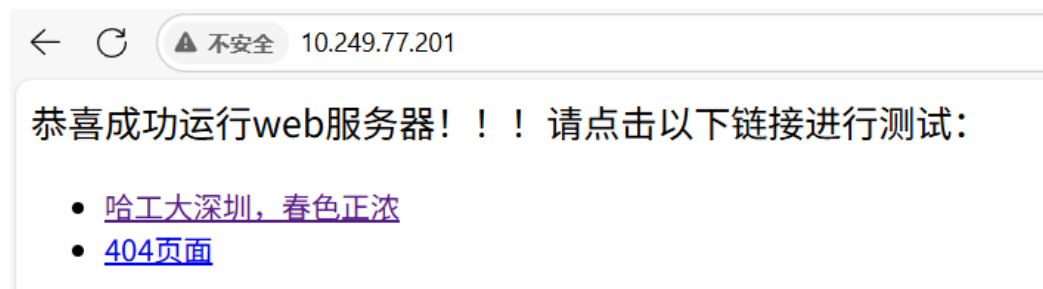
7. web 服务器实验结果及分析

(展示 web 服务器的请求和响应过程截图, 分析 HTTP 请求和响应的格式、内容。检查请求方法、请求 URL、请求头、响应状态码、响应头、响应体等部分。)

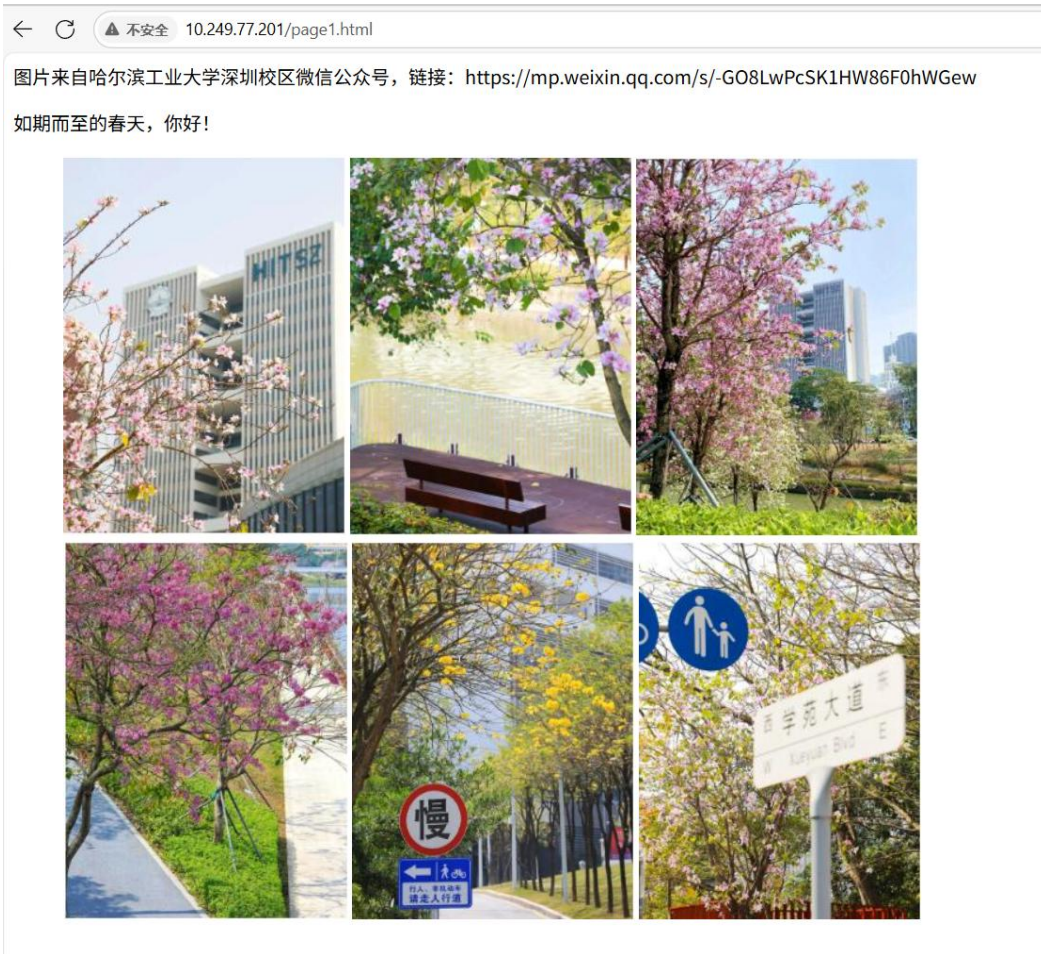
对于 web_server, 在终端中运行 main:

```
(dl) PS C:\Users\Jin_zd\Desktop\Code\Protocol-Stack-Lab\build> . "C:\Users\Jin_zd\Desktop\Code\Protocol-Stack-Lab\build\web_server.exe"
Using interface \Device\NPF_{6F607EFA-327D-4091-A9A4-6F6228C84BDF}, my ip is 10.249.77.201.
```

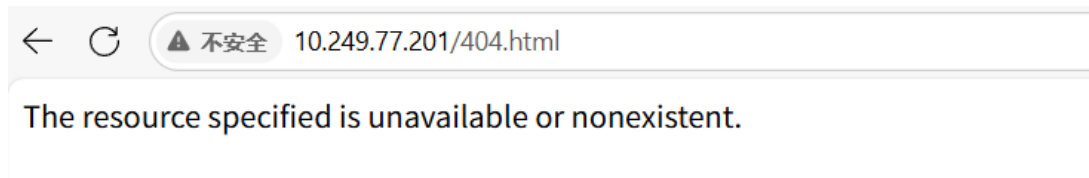
在浏览器中访问 10.249.77.201:



点击第一个链接, 获得响应:



点击第二个链接，获得响应：



使用 Wireshark 在筛选 `eth.addr==11:22:33:44:55:66`（即 10.249.77.201 的 MAC 地址）条件下的报文：

No.	Time	Source	Destination	Protocol	Length	Info
287	45.163404	10.249.77.201	10.249.77.200	TCP	1078	80 → 2707 [ACK] Seq=66455 Ack=2286 Win=65535 Len=0 [TCP PDU reassembled in 290]
288	45.163431	10.249.77.200	10.249.77.201	TCP	54	2707 → 80 [ACK] Seq=2286 Ack=67479 Win=65535 Len=0
289	45.163452	10.249.77.201	10.249.77.200	TCP	1078	80 → 2707 [ACK] Seq=67479 Ack=2286 Win=65535 Len=0 [TCP PDU reassembled in 290]
290	45.163531	10.249.77.201	10.249.77.200	HTTP	1055	HTTP/1.1 200 OK (JPEG JFIF image)
291	45.163563	10.249.77.201	10.249.77.201	TCP	54	2707 → 80 [ACK] Seq=2286 Ack=69504 Win=65535 Len=0
292	45.215983	10.249.77.200	10.249.77.201	TCP	54	2845 → 80 [ACK] Seq=1294 Ack=69837 Win=65098 Len=0
293	71.136585	10.249.77.200	10.249.77.201	HTTP	569	GET /404.html HTTP/1.1
294	71.136735	10.249.77.201	10.249.77.200	TCP	78	80 → 2707 [ACK] Seq=69504 Ack=2801 Win=65535 Len=24 [TCP PDU reassembled in 301]
295	71.136839	10.249.77.201	10.249.77.200	TCP	78	80 → 2707 [ACK] Seq=69528 Ack=2801 Win=65535 Len=24 [TCP PDU reassembled in 301]
296	71.136868	10.249.77.200	10.249.77.201	TCP	54	2707 → 80 [ACK] Seq=2801 Ack=69552 Win=65487 Len=0
297	71.136898	10.249.77.201	10.249.77.200	TCP	94	80 → 2707 [ACK] Seq=69552 Ack=2801 Win=65535 Len=40 [TCP PDU reassembled in 301]
298	71.136929	10.249.77.201	10.249.77.200	TCP	75	80 → 2707 [ACK] Seq=69592 Ack=2801 Win=65535 Len=21 [TCP PDU reassembled in 301]
299	71.136947	10.249.77.200	10.249.77.201	TCP	54	2707 → 80 [ACK] Seq=2801 Ack=69613 Win=65426 Len=0
300	71.136963	10.249.77.201	10.249.77.200	TCP	60	80 → 2707 [ACK] Seq=69613 Ack=2801 Win=65535 Len=2 [TCP PDU reassembled in 301]
301	71.136991	10.249.77.201	10.249.77.200	HTTP	158	HTTP/1.1 404 Not Found (text/html)
302	71.137008	10.249.77.200	10.249.77.201	TCP	54	2707 → 80 [ACK] Seq=2801 Ack=69719 Win=65320 Len=0
315	90.162186	10.249.77.200	10.249.77.201	TCP	55	[TCP Keep-Alive] 2846 → 80 [ACK] Seq=0 Ack=1 Win=65535 Len=1
316	90.162271	10.249.77.200	10.249.77.201	TCP	55	[TCP Keep-Alive] 2847 → 80 [ACK] Seq=0 Ack=1 Win=65535 Len=1
317	90.162287	10.249.77.200	10.249.77.201	TCP	55	[TCP Keep-Alive] 2845 → 80 [ACK] Seq=1293 Ack=69837 Win=65098 Len=1
318	90.162325	10.249.77.201	10.249.77.200	TCP	60	[TCP Keep-Alive ACK] 80 → 2846 [ACK] Seq=1 Ack=1 Win=65535 Len=0
319	90.162389	10.249.77.201	10.249.77.200	TCP	60	[TCP Keep-Alive ACK] 80 → 2847 [ACK] Seq=1 Ack=1 Win=65535 Len=0
320	90.162410	10.249.77.201	10.249.77.200	TCP	60	[TCP Keep-Alive ACK] 80 → 2845 [ACK] Seq=69837 Ack=1294 Win=65535 Len=0
321	116.138495	10.249.77.200	10.249.77.201	TCP	55	[TCP Keep-Alive] 2707 → 80 [ACK] Seq=2800 Ack=69719 Win=65320 Len=1
322	116.138531	10.249.77.201	10.249.77.200	TCP	60	[TCP Keep-Alive ACK] 80 → 2707 [ACK] Seq=69719 Ack=2801 Win=65535 Len=0
323	134.003560	10.249.77.200	10.249.77.201	TCP	54	2846 → 80 [FIN, ACK] Seq=1 Ack=1 Win=65535 Len=0
324	134.003592	10.249.77.201	10.249.77.200	TCP	60	80 → 2846 [FIN, ACK] Seq=1 Ack=2 Win=65535 Len=0
325	134.003620	10.249.77.201	10.249.77.201	TCP	54	2847 → 80 [FIN, ACK] Seq=1 Ack=1 Win=65535 Len=0
326	134.003674	10.249.77.200	10.249.77.201	TCP	54	2846 → 80 [ACK] Seq=2 Ack=2 Win=65535 Len=0
327	134.003733	10.249.77.201	10.249.77.200	TCP	60	80 → 2847 [FIN, ACK] Seq=1 Ack=2 Win=65535 Len=0
328	134.003768	10.249.77.200	10.249.77.201	TCP	54	2847 → 80 [ACK] Seq=2 Ack=2 Win=65535 Len=0
329	135.165565	10.249.77.200	10.249.77.201	TCP	55	[TCP Keep-Alive] 2845 → 80 [ACK] Seq=1293 Ack=69837 Win=65098 Len=1
330	135.165593	10.249.77.201	10.249.77.200	TCP	60	[TCP Keep-Alive ACK] 80 → 2845 [ACK] Seq=69837 Ack=1294 Win=65535 Len=0

分析第 290 条报文，即 HTTP 响应报文：

> Frame 290: 1055 bytes on wire (8440 bits), 1055 bytes captured (8440 bits) on interface \Device\NPF... (6f6)	0000	48 54 50 2f 31 2e 11 20 32 30 30 20 4f db 00	HTTP/1.1 200 OK
> Ethernet II, Src: 11:22:33:44:55:66 (11:22:33:44:55:66), Dst: Microsoft_A4:66:01 (00:15:5d:4d:66:01)	0010	0a 43 6f 6e 05 63 7a 09 f4 6a 3a 20 4b 65 65	(Connect ion: Kee
> Internet Protocol Version 4, Src: 10.249.77.201, Dst: 10.249.77.200	0020	70 2d 41 6c 09 76 65 0d 0a 43 6f 6a 74 65 6e 74	p-Alive: .Content
> Transmission Control Protocol, Src Port: 80, Dst Port: 2707, Seq: 68503, Ack: 2286, Len: 1001	0030	2d 54 79 70 05 3a 20 69 6d 61 67 65 2f 6a 70 65	-Type: i mage/jpe
> [30 Reassembled TCP Segments (25669 bytes): #247(17), #248(24), #250(26), #251(23), #253(2), #254(1024),	0040	07 0d 0a 43 6f 6e 74 65 6e 74 2d 4c 05 6e 02 7d	g .Conte nt-Lengi
> Hypertext Transfer Protocol	0050	0a 3a 20 32 35 35 37 37 0d 0a 0d 0a ff db ff e0	0: 25577 ...
> HTTP/1.1 200 OK\r\n	0060	00 10 4a 46 49 46 00 01 01 01 00 60 60 60 00	...JFIF
Response Version: HTTP/1.1	0070	ff db 00 43 00 08 06 06 07 06 05 00 07 07 09	...C.....
Status Code: 200	0080	09 08 0a 0c 14 0d 0c 0b 0b 0c 19 12 13 0f 14 1d	
[Status Code Description: OK]	0090	1a 1f 1e 1d 1a 1c 1c 20 24 26 27 20 22 2c 23 1c	\$. " ,#
Response Phrase: OK	00a0	1c 28 37 29 2c 30 31 34 34 34 1f 27 39 3d 38 32	...(7),014 44:'9=82
Connection: Keep-Alive\r\n	00b0	3c 2e 33 34 32 ff db 00 43 01 09 09 09 0c 0b 0c	...<342...C.....
Content-Type: image/jpeg\r\n	00c0	1b 0d 0d 18 32 11 1c 21 32 32 32 32 32 32 32	...211 22222222
Content-Length: 25577\r\n	00d0	32 32 32 32 32 32 32 32 32 32 32 32 32 32 32	22222222 22222222
[Content length: 25577]	00e0	32 32 32 32 32 32 32 32 32 32 32 32 32 32 32	22222222 22222222
\r\n	00f0	32 32 32 32 32 32 32 32 32 32 ff c2 00 11 08 01	22222222 22.....
[Request in frame: 199]	0100	40 00 f0 03 01 22 00 02 11 01 03 11 01 ff c4 00	@ ".....
[Time since request: 0.005028800 seconds]	0110	1b 00 00 02 03 01 01 01 01 00 00 00 00 00 00	
[Request URI: /assets/img6.jpg]	0120	00 00 03 04 01 02 05 06 00 07 ff c4 00 19 01 00	
[Full request URI: http://10.249.77.201/assets/img6.jpg]	0130	03 01 01 01 00 00 00 00 00 00 00 00 00 00 00	
File Data: 25577 bytes	0140	01 02 03 04 05 ff da 00 0c 03 01 00 02 10 03 10	
> JPEG File Interchange Format	0150	00 00 01 ef fd ef 00 ef 23 0a 81 c5 82 43 7a 21	#.....Cz!
	0160	c4 41 a2 cc c6 90 02 35 ea d5 01 90 35 4e 36 22	...A.....S.....5N6"
	0170	b0 d2 69 3f 0e c8 b3 19 ec da 7d 35 b5 c4 a2 8d	...p.....35.....
	0180	4b 16 0a c6 54 c8 15 34 9d 34 2c bd 4e ad 13 3b	...K...T...4...A...N...;
	0190	22 fe 02 3c 18 5a 5e ce 25 34 64 5d 46 f2 ed 75	..."-<Z"- %dJF--u
	01a0	34 4a 08 20 c4 a9 ad 20 a4 e8 b1 d8 3b bc d5 65	adD...T...

由图可知，HTTP 响应信息：

HTTP/1.1 200 OK

响应版本: HTTP/1.1

状态码: 200

状态码描述: OK

响应短语: OK

连接: Keep-Alive (保持连接)

内容类型: image/jpeg (图片/jpeg)

内容长度: 25577

请求位于帧: 199

自请求以来的时间: 0.005028800 秒

请求 URI: /assets/img6.jpg

完整请求 URI: <http://10.249.77.201/assets/img6.jpg>

文件数据: 25577 字节

分析第 293 条报文，即 HTTP 请求报文：

> Frame 293: 569 bytes on wire (4552 bits), 569 bytes captured (4552 bits) on interface \Device\NPF_{6F6D7000-0010-0000-8000-000000000000} (00:15:5d:4d:66:01)		0000	11 22 33 44 55 66 00 15	5d 4d 66 01 08 00 45 00	"30U...]MF...E-
> Ethernet II, Src: Microsoft_Ad:66:01 (00:15:5d:4d:66:01), Dst: 11:22:33:44:55:66 (11:22:33:44:55:66)		0010	02 2b ac 86 40 00 80 06	9a c3 0a f9 4d c8 0a f9	+...@... ..M...
> Internet Protocol Version 4, Src: 10.249.77.200, Dst: 10.249.77.201		0020	4d c9 0a 93 00 50 83 71	7f 5a 00 01 83 96 50 18	M...P.q.Z...P
> Transmission Control Protocol, Src Port: 2707, Dst Port: 80, Seq: 2286, Ack: 69504, Len: 515		0030	ff ff 6c c5 00 00 17 43	54 20 2f 34 30 54 2e 00	.1...T...40424
> Hypertext Transfer Protocol		0040	74 6d 6c 20 49 54 54 50	2f 31 2e 31 0d 0a 48 6f	[a] HTTP /1.1: No
GET /404.html HTTP/1.1\r\n		0050	73 74 3a 20 31 30 2e 32	34 39 2e 37 37 2e 32 30	st: 10.2.49.77.20
Request Method: GET		0060	31 0d 0a 43 6f 6e 6e 65	63 74 69 6f 6e 3a 20 6b	1-Connec tion: k
Request URI: /404.html		0070	65 65 70 2d 61 6c 69 76	65 0d 0a 55 70 67 72 61	keep-aliv e->Upgr
Request Version: HTTP/1.1		0080	64 65 2d 49 6e 73 65 63	75 72 65 2d 52 65 71 75	de-Insec ure-Keo
Host: 10.249.77.201\r\n		0090	65 73 74 73 3a 20 31 0d	0a 55 73 65 72 2d 41 67	ests: 1-User-Ag
Connection: keep-alive\r\n		00a0	65 6e 74 3a 20 4d 6f 7a	69 6c 6c 61 2f 35 2e 30	ent: Moz illa/5.0
Upgrade-Insecure-Requests: 1\r\n		00b0	20 28 57 69 6e 64 6f 77	72 20 4e 54 20 31 30 2e	(Window s NT 10.0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/119.0.0.0 Safari/537.36		00c0	30 3b 20 57 69 6e 36 34	3b 20 78 36 34 29 20 41	p; WinGd ; x64) A
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8		00d0	70 70 6c 65 57 65 62 4b	69 74 2f 35 33 37 2e 33	ppleWebK it/537.3
Referer: http://10.249.77.201/\r\n		00e0	36 20 28 4b 48 54 4d 4c	2c 20 6c 69 6b 65 20 47	6 (KHTML , like G
Accept-Encoding: gzip, deflate\r\n		00f0	65 63 6b 6f 29 20 43 68	72 6f 6d 65 2f 31 33 36	cko) Ch rom/119
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-US;q=0.7		0100	2e 30 2e 30 2e 30 20 53	61 66 61 72 69 2f 35 33	0.0.0.0 S afari/53
\r\n		0110	37 2e 33 36 20 45 64 67	2f 31 33 36 2e 30 2e 30	7.36 Edg /136.0.0
[Response in frame: 301]		0120	2e 30 0d 0a 41 63 63 65	70 74 3a 20 74 65 78 7d	0-:Acce pt: text
[Full request URI: http://10.249.77.201/404.html]		0130	2f 68 74 6d 6c 2c 61 70	70 6c 69 63 61 74 69 6f	/html,ap plicatio
		0140	6e 2f 78 68 74 6d 6c 2b	70 6d 6c 2c 61 70 70 6c	/a/html, xml,appl
		0150	69 63 61 74 69 6f 6e 2f	78 6d 6c 3b 71 3d 30 2e	ication/ xml;q=0
		0160	39 2c 69 6d 61 67 65 2f	61 76 69 66 2c 69 6d 61	9,image/ avif,ima
		0170	67 65 2f 77 65 62 70 2c	69 6d 61 67 65 2f 61 70	ge/webp, image/ap
		0180	6e 67 2c 2a 2f 2a 3b 71	3d 30 2e 38 2c 61 70 70	ng,/*;q=0.8,app
		0190	6c 69 63 61 74 69 6f 6e	2f 73 69 67 6e 65 64 2d	lication /signed
		01a0	65 78 63 68 61 6e 67 65	3b 76 3d 62 33 3b 71 3d	exchange ;v=b3;q
		01b0	30 2e 37 0d 0a 52 65 66	65 72 65 72 3a 20 68 7d	0.7-:Ref erer: ht
		01c0	7d 70 3a 2f 2f 31 30 2e	32 34 39 2e 37 37 2e 32	tp://10. 249.77.2
		01d0	30 31 2f 0d 0a 41 63 63	65 70 74 2d 45 6e 63 6f	01/-Acc pt-Ince
		01e0	64 69 6e 67 3a 20 67 7a	69 70 2c 20 64 65 66 6c	ding: gz ip, defl
		01f0	61 74 65 0d 0a 41 63 63	65 70 74 2d 4c 61 6e 67	ate-Acc ept-Lang
		0200	75 61 67 65 3a 20 7a 68	2d 43 4e 2c 7a 68 3b 71	uage: zh -Gn,zh,en
		0210	3d 30 2e 39 2c 65 6e 3b	71 3d 30 2e 38 2c 65 6e	=0.9,en; q=0.8,en
		0220	2d 47 42 3b 71 3d 30 2e	37 2c 65 6e 2d 55 53 3b	-GB;q=0. 7,en-US;
		0230	71 3d 30 2e 36 0d 0a 0d 00		q=0.6...

由图可知，HTTP 请求信息：

GET /404.html HTTP/1.1

请求方法: GET

请求 URI: /404.html

请求版本: HTTP/1.1

主机: 10.249.77.201

连接: Keep-Alive (保持连接)

Upgrade-Insecure-Requests: 1

用户代理: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like

Gecko) Chrome/119.0.0.0 Safari/537.36

接受 text/html,application/xhtml+xml...

引用: http://10.249.77.201/

接受编码: gzip, deflate (gzip, deflate)

接受语言: zh-CN,zh;q=0.9,en;q=0.8,zh-TW;q=0.7,en-US;q=0.6

响应位于帧: 301

完整请求 URI: http://10.249.77.201/404.html

分析第 301 条报文，即 HTTP 响应报文：

> Frame 301: 158 bytes on wire (1264 bits), 158 bytes captured (1264 bits) on interface \Device\NPF_{6F6D7000-0010-0000-8000-000000000000} (00:15:5d:4d:66:01)		0000	48 54 54 50 2f 31 2e 31	20 34 30 34 20 4e 6f 74	HTTP/1.1 404 Not
> Ethernet II, Src: 11:22:33:44:55:66 (11:22:33:44:55:66), Dst: Microsoft_Ad:66:01 (00:15:5d:4d:66:01)		0010	20 46 6f 75 6e 64 0d 0a	43 6f 6e 6e 65 63 74 65	Found- Connec ti
> Internet Protocol Version 4, Src: 10.249.77.200, Dst: 10.249.77.201		0020	6f 6e 3a 20 4b 65 65 70	2d 41 6c 69 70 65 0d 0a	on: Keep-Aliv e-
> Transmission Control Protocol, Src Port: 80, Dst Port: 2707, Seq: 69515, Ack: 2801, Len: 104		0030	43 6f 6e 74 65 6e 74 2d	54 79 70 65 3a 20 74 65	Content- Type: te
> [6 Reassembled TCP Segments (215 bytes): #294(24), #295(24), #297(40), #298(21), #300(2), #301(104)]		0040	78 74 2f 68 74 6d 6c 3b	20 63 68 61 72 73 65 74	xt/html; charset
> Hypertext Transfer Protocol		0050	8d 75 74 66 20 38 0d 0a	43 6f 6e 74 65 6e 7a 2d	utf-8- Content-
HTTP/1.1 404 Not Found\r\n		0060	4c 65 6e 67 74 68 3a 20	31 30 3a 0d 0a 0d 0a 3c	length: 104...
Response Version: HTTP/1.1		0070	48 54 4d 4c 3e 3c 54 49	54 4c 45 3e 4e 6f 74 20	HTML<T TLE>Not
Status Code: 404		0080	46 6f 75 6e 64 3c 2f 54	49 54 4c 45 3e 0d 0a 54	Found<T TLE>.T
[Status Code Description: Not Found]		0090	68 65 20 72 65 73 6f 75	72 63 65 20 73 70 65 63	he resou rce spec
Response Phrase: Not Found		00a0	69 66 69 65 64 0d 0a 69	73 20 75 6e 61 70 61 69	ified- i s unava
Connection: Keep-Alive\r\n		00b0	6c 61 62 6c 65 20 6f 72	20 6e 6f 6e 65 78 69 73	lable or nonexis
Content-Type: text/html; charset=utf-8\r\n		00c0	74 65 6e 74 2e 0d 0a 3c	2f 42 4f 4d 49 3e 3c 2f	tent...< /BODY></
Content-Length: 104\r\n		00d0	48 54 4d 4c 3e 0d 0a 00		HTML>...
[Content length: 104]					
\r\n					
[Request in frame: 293]					
[Time since request: 0.000406000 seconds]					
[Request URI: /404.html]					
[Full request URI: http://10.249.77.201/404.html]					
File Data: 104 bytes					
> Line-based text data: text/html (4 lines)					

由图可知，HTTP 响应信息：

HTTP/1.1 404 Not Found

响应版本: HTTP/1.1

状态码: 404

状态码描述: Not Found

响应短语: Not Found

连接: Keep-Alive
内容类型: text/html; charset=utf-8 (文本/HTML; 字符集=utf-8)
内容长度: 104
请求位于帧: 293
自请求以来的时间: 0.000400000 秒
请求 URI: /404.html
完整请求 URI: http://10.249.77.201/404.html
文件数据: 104 字节
基于行的文本数据: text/html (4 行)

上述分析的 HTTP 报文信息均符合预期, 表明 HTTP 协议相关函数实现正确

三、 实验中遇到的问题及解决方法

(详细描述在设计或测试过程中遇到的问题, 包括错误描述、排查过程以及最终的解决方案。)

问题:

在完成以太网协议后, 在测试时发现测试不通过, 检查 log 文件, 发现目标设备接收到的数据包内容不正确。

排查过程:

检查 MAC 地址设置: 确认发送端和接收端的 MAC 地址是否正确配置, 并且目标设备的 MAC 地址是否与以太网帧的目的 MAC 地址匹配。

Wireshark 抓包分析: 在发送端, 确认 Wireshark 抓到的数据帧是否与程序发送的数据一致。重点检查数据部分的长度和内容。在接收端, 确认 Wireshark 是否能抓到数据帧, 以及抓到的数据帧内容是否正确。

字节序: 发送端和接收端的处理器字节序可能不同(大端序或小端序)。如果数据包中包含多字节的数值(如长度、ID 等), 并且没有进行字节序转换, 接收端解析时会得到错误的值。

解决方案:

发现问题在于发送端和接收端在处理数据包中的多字节数值时, 没有将需要转换的字段进行的字节序转换。发送端使用的是网络字节序(大端序), 而接收端平台(x86)期望的是小端序。

问题:

在完成 IP 协议实验后, 发现 IP 分片测试不能通过。

排查过程:

确认 MTU: 确保 ip_out 函数的 MTU 大小设置正确, 没有将 IP 头部大小包含在内。

IP 头字段检查:

标识字段: 确认分片的 IP 数据包的标识字段是否一致。

标志字段: 检查 MF 标志位是否在除了最后一个分片外都正确设置。

片偏移: 检查每个分片的片偏移是否正确计算, 表示该分片数据在原始 IP 数据报中的起始位置。

总长度: 确认每个分片的总长度字段是否正确。

解决方案:

发现问题在于标识字段的设置, 第一个数据报的标识必须设置为 0, 而后每送一个完整的数据报加一。测试只检测第一个数据报的标识是不是 0, 可能是测试一共就测试了一个长数据报。从其他数字开始的标识一律报错。

问题:

在完成 UDP 实验之后, 进行 `udp_server` 测试时, 发现只有第一个数据包能被正常接受, 后续数据包均无法获得响应。

排查过程:

UDP 协议检查: 通过 Wireshark 抓包, 获取发送 UDP 头部字段, 检查源端口地址、目的端口地址、长度、校验和、载荷数据是否都正确。

IP 协议检查: 通过 Wireshark 抓包, 检查 IP 头部字段, 检查源 IP 地址和目的 IP 地址是否正确, 检查校验和是否正确。

ARP 报文检查: 检查 ARP 缓存逻辑是否正确, 重点通过 Wireshark 抓包检查第一次发送 ARP 请求时的报文情况, 只有第一次发送数据包会被缓存, 等待获取 MAC 地址后再发送。

解决方法:

发现问题在于无 ARP 缓存时, 将待发送数据包暂存, 获得 MAC 地址后, 调用 `eth_out` 将数据包以 IP 协议发送, 而之后的数据包发送, 命中 ARP 缓存的情况, 调用 `eth_out` 是按照 ARP 协议将数据包发送出去的, 所以收不到这些数据包。将命中 ARP 缓存的情况也改为以 IP 协议调用 `eth_out` 将数据包发送即可。

四、实验收获和建议

(总结配置实验及协议栈实验过程中的实践收获, 结合实操体验针对性提出实验流程优化及环境完善建议, 为后续实验教学与研究的迭代改进提供参考依据。)

实验收获:

- Lab1 (VLAN 与接口模式配置) 使我能够熟练地在网络设备上配置 VLAN, 理解了 VLAN 隔离原理和不同端口模式 (Access/Trunk) 的应用场景, 让我对局域网内部的通信机制有了更清晰的认识。
- 从 Lab2 (Eth 协议实现) 到 Lab7 (UDP 协议实现), 我亲手从零开始实现了 TCP/IP 协议栈中的关键协议。
- Eth 与 ARP 层: 我掌握了以太网帧的封装与解封装过程, 通过实现 ARP, 深刻理解了 IP 地址与 MAC 地址之间的映射关系, 以及 ARP 请求/应答报文的交互流程。
- IP 层: 我学会了 IP 数据报的封装、分片与重组等核心机制, 特别是对 IP 校验和的计算和验证, 让我对数据传输的可靠性有了更直观的感受。
- ICMP 与 UDP 层: 通过实现 ICMP, 我理解了网络控制消息的作用, 如 Echo 请求/应答、目标不可达等。UDP 的实现则让我体会到无连接传输的特点及其在特定应用场景下的优势。

- 在整个过程中，我清晰地认识到各层协议并非独立工作，而是紧密协作，共同完成数据传输任务。例如，IP 层依赖于 ARP 获取下一跳 MAC 地址，而 Ethernet 层负责将 IP 数据报承载并通过物理介质发送。
- Lab4（RIP 路由配置及协议分析）让我首次接触并配置了动态路由协议，通过 Wireshark 抓包分析，我对 RIP 路由信息的交换过程和路由表的生成原理有了直观的认识。
- Lab8（IPv6 网络过渡实验）则补充了我在 IPv6 方面的知识空白，实践了隧道过渡技术。
- Lab9（小型 Web 服务器的设计与实现）要求我将前期所学网络协议知识融会贯通。在此实验中，我实现了 TCP 协议的连接管理与可靠传输机制，更在此基础上实现了简单的 HTTP/1.0 协议，理解了 Web 通信流程。
- 在实验过程中，我遇到了大量的 bug 和问题，如协议头字段错误、校验和计算错误、缓冲区溢出等。通过使用 Wireshark 进行协议分析、GDB 进行代码调试我的独立问题排查和解决能力得到了极大锻炼。尤其是在一些协议实现不够完善导致后续问题暴露时，反复的排查和定位过程让我更加深刻地理解了协议的细节和复杂性。

完善建议：

- 在本次实验中，我遇到在 UDP 协议实现（Lab7）时，发现之前 Eth 协议（Lab2）或 ARP 协议（Lab3）存在微小 bug，导致 UDP 数据包无法正确收发。这使得问题定位非常困难，耗费了大量时间和精力。往往一个底层协议的细微错误就会影响上层协议的正确性，并且很难溯源。
- 亟须引入更完善强制性的单元测试：在每个协议（Eth, ARP, IP, ICMP, UDP, TCP）实现完成后，建议增加覆盖面更广的自动化单元测试用例，并且需要明确的输出错误原因。除了基本的数据包发送接收，应严格测试各个协议层的特有的功能。每个子功能都应有独立的测试点。并且测试点不应该只有几个，需要多种情况，涵盖不同的边界。
- 建议使用统一的虚拟机镜像或 Docker 容器，预装所有必要的开发工具（GCC, CMake 等）、调试工具（GDB）、网络分析工具（Wireshark）。实验指导书可以加入关于虚拟网卡的详细配置步骤。这可以避免因环境配置问题耗费过多时间，确保实验环境的一致性。
- 在实验指导书中，除了理论部分，可以增加更多关于常见错误的排查思路和调试技巧的指导。配置实验中可以加入更多对指令的解释，不然我们只能机械的复制运行代码。